Contents

Calling snowflake stored proc from power bi:

https://docs.snowflake.com/en/developer-guide/stored-procedure/stored-procedures-selecting-from

case 1 : proc with out ddl and dml

Snowflake proc sample:

```
CREATE OR REPLACE PROCEDURE RPT.TEST_VMAC_NEW()
RETURNS TABLE (
SRC_AUTH_ID VARCHAR,PCP_ID VARCHAR ,AUTH_CREATE_DATE DATE , CDM_CREATE_DT TIMESTAMP_NTZ(9)
)
LANGUAGE SQL
EXECUTE AS CALLER
AS
$$
DECLARE
res RESULTSET;
BEGIN
res:= (
SELECT  SRC_AUTH_ID,
PCP_ID,
AUTH_CREATE_DATE,
CDM_CREATE_DT
FROM dbo.Auth_Line_Banners LIMIT 10
);
RETURN TABLE(res);
END;
$$
;

call  TEST_VMAC_NEW ();

select * from (TABLE(CDP_QA.RPT.TEST_VMAC_NEW ()));
```

sample_proc_call_from_pbi.sql

Select * from (table (cdp_aq.rpt.TEST_VMAC_NEW()) .. we need to use this query in power bi to get the results from snowflake procedure.

Power BI desktop :

Connection settings

You can choose how to connect to this data source. Import allows you to bring a copy of the data into Power BI. DirectQuery will connect live to this data source.

- ○ Import
- ◉ DirectQuery

Learn more about DirectQuery

OK Cancel

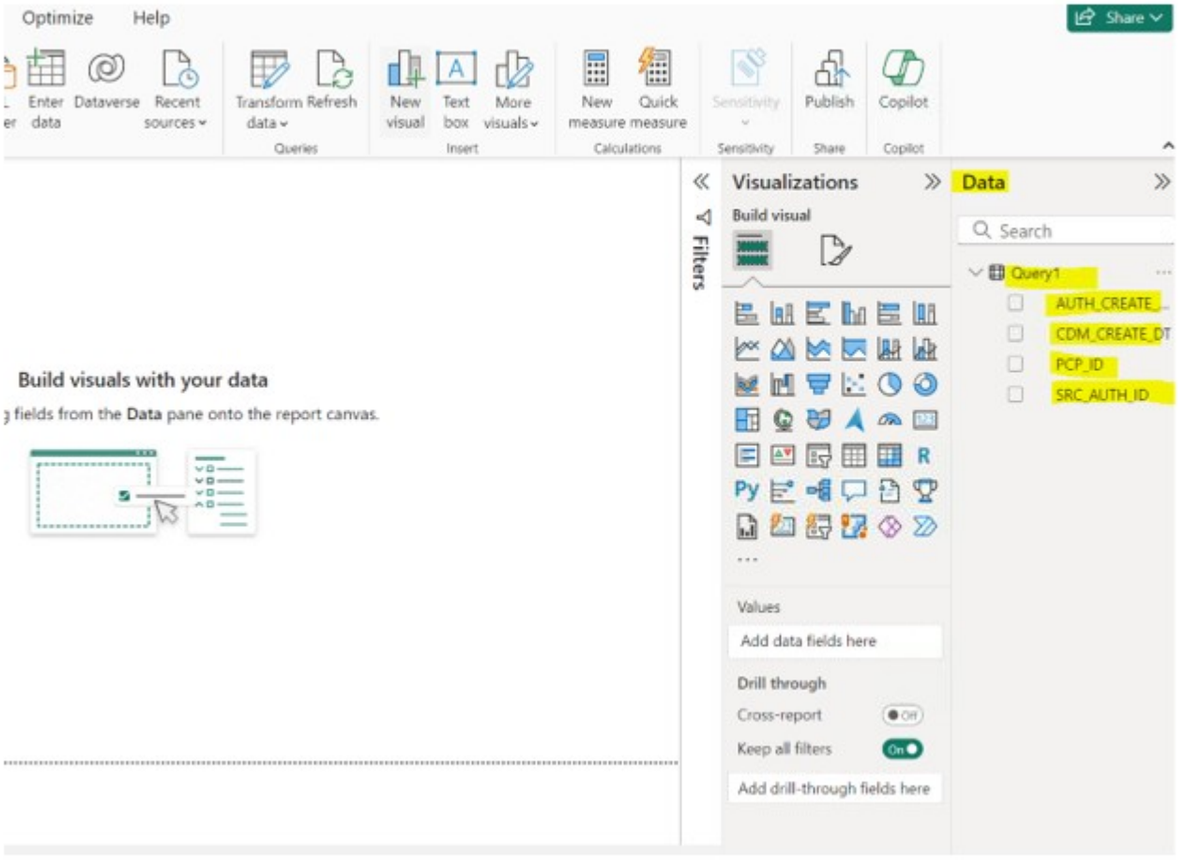
humana-az_snowflake_eastus2.privatelink.snowflakecomputing.com:CDP_COMMON_W...

SRC_AUTH_ID	PCP_ID	AUTH_CREATE_DATE	CDM_CREATE_DT
000000000	820585205G	5/23/2012	3/23/2023 10:19:32 AM
088024521	000210735C	11/16/2016	3/23/2023 10:19:32 AM
088032112	000067268J	2/9/2017	3/23/2023 10:19:32 AM
088045543	null	2/9/2018	3/23/2023 10:19:32 AM
088048725	561935767D	4/27/2018	3/23/2023 10:19:32 AM
088089294	000088729G	8/29/2018	3/23/2023 10:19:32 AM
088094867	000189856G	9/4/2018	3/23/2023 10:19:32 AM
088095053	000189856G	9/4/2018	3/23/2023 10:19:32 AM
088170352	000172028	3/4/2019	3/23/2023 10:19:32 AM
088209176	000269033T	10/15/2019	3/23/2023 10:19:32 AM

Load Transform Data Cancel

Click on Ok

We can see the query o/p columns in DATA pane in below image



Case 2 : strored proc with ddl and dml scenario

CASE 2 : Snowflake proc sample: with DDL and DML in procedure

```
CREATE OR REPLACE PROCEDURE RPT.TEST_VMAC_NEW_DDL_DML()
RETURNS TABLE (
SRC_AUTH_ID VARCHAR,PCP_ID VARCHAR ,AUTH_CREATE_DATE DATE , CDM_CREATE_DT TIMESTAMP_NTZ(3)
)
LANGUAGE SQL
EXECUTE AS CALLER
AS
$$
DECLARE
res RESULTSET;
BEGIN
CREATE  OR REPLACE TEMP TABLE TEST_DATA
AS
SELECT SRC_AUTH_ID,
PCP_ID,
AUTH_CREATE_DATE,
CDM_CREATE_DT
FROM dbo.Auth_Line_Banners LIMIT 10
;
res:= (
SELECT  SRC_AUTH_ID,
PCP_ID,
AUTH_CREATE_DATE,
CDM_CREATE_DT
FROM TEST_DATA
);

RETURN TABLE(res);
END;
$$
;
```

If we call this procedure as part of select statement it will throw error :

```
38
39 select * from (TABLE(CDP_QA.RPT.TEST_VMAC_NEW_DDL_DML ()));
```

Results Data Preview Open History

✖ Query ID SQL 986ms

Stored procedure execution error on line 1 position 21: Uncaught exception of type 'STATEMENT_ERROR' on line 8 at position 0 : SQL compilation error: Invalid statement type 'CREATE' for child job from stored procedure in FROM clause on TABLE 'TEST_DATA'.

Note :As per snowflake documentation:

- Only stored procedures that perform SELECT, SHOW, DESCRIBE, or CALL statements can be placed in the FROM clause of a SELECT statement. Stored procedures that make modifications using DDL or DML operations aren't allowed. For stored procedures that issue CALL statements, these limitations apply to the stored procedures that are called.

If the procedure has ddl or dml then we cant call that procedure as part of select statement .. so we cant use that proc from select statement in power Bi also.

External tables

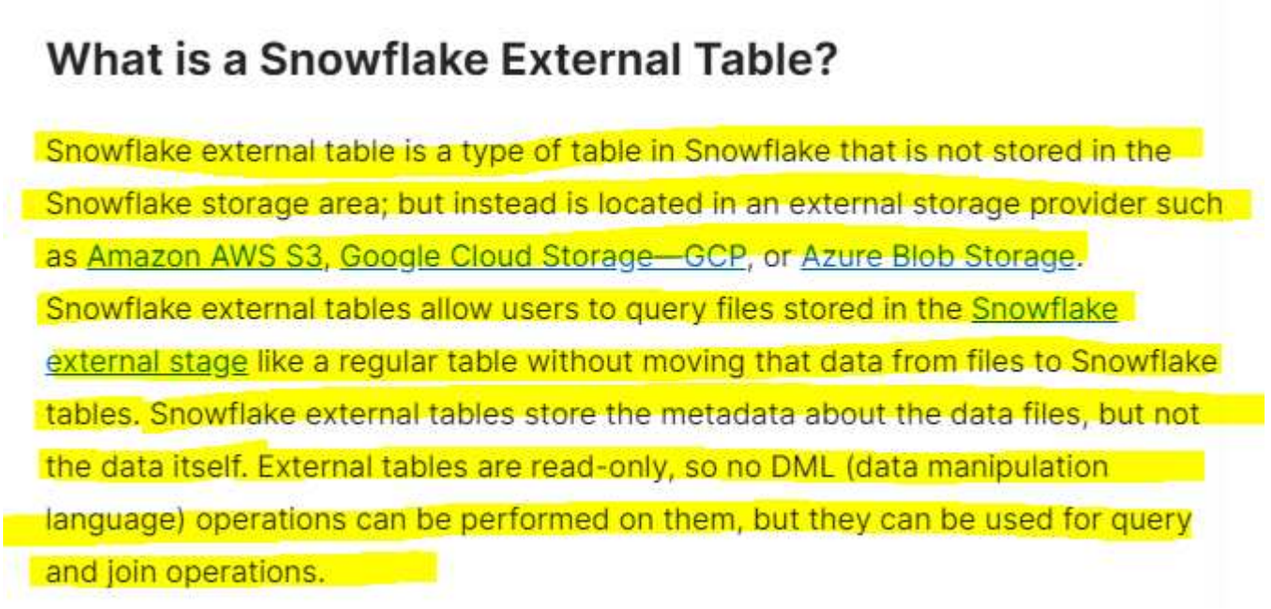
https://interworks.com/blog/2023/09/19/snowflake-external-tables-connect-efficiently-to-your-data-lake/

refer ch Pradeep video from udemy

What is a Snowflake External Table?

- Snowflake external table is a type of table in Snowflake that is not stored in the Snowflake storage area; but instead is located in an external storage provider such as Amazon AWS S3, Google Cloud Storage—GCP, or Azure Blob Storage.

- Snowflake external tables allow users to query files stored in the Snowflake external stage like a regular table without moving that data from files to Snowflake tables.
- Snowflake external tables store the metadata about the data files, but not the data itself
- External tables are read-only, so no DML (data manipulation language) operations can be performed on them
- they can be used for query and join operations.



What is a Snowflake External Table?

Snowflake external table is a type of table in Snowflake that is not stored in the Snowflake storage area; but instead is located in an external storage provider such as Amazon AWS S3, Google Cloud Storage—GCP, or Azure Blob Storage. Snowflake external tables allow users to query files stored in the Snowflake external stage like a regular table without moving that data from files to Snowflake tables. Snowflake external tables store the metadata about the data files, but not the data itself. External tables are read-only, so no DML (data manipulation language) operations can be performed on them, but they can be used for query and join operations.

Advantages of Snowflake external tables

- Snowflake external tables allow analyzing data without storing it in Snowflake.
- Querying data from Snowflake external tables is possible without moving data from files to Snowflake, saving time and storage space.
- Snowflake external tables provide a way to query multiple files by joining them into a single table.
- Snowflake external tables support query and join operations and can be used to create views, security, and materialized views.

Disadvantages of Snowflake external tables

- Querying data from Snowflake external tables is slower than querying data from internal tables.
- Snowflake external tables are read-only, so DML operations cannot be performed on them.
- Snowflake external tables require a Snowflake external stage to be set up, which can add complexity to the system.

What are the key features of Snowflake external tables?

- Snowflake external tables are not stored in the Snowflake storage area but in external storage providers (AWS, GCP, or Azure).
- Snowflake external tables allow querying files stored in Snowflake external stages like regular tables without moving the data from files to Snowflake tables.
- Snowflake external tables access the files stored in the Snowflake external stage area, such as AWS S3, GCP Bucket, or Azure Blob Storage.
- Snowflake external tables store metadata about the data files.
- Snowflake external tables are read-only so that no DML operations can be performed.
- Snowflake external tables support query and join operations and can be used to create views, security, and materialized views.

Diff b/w normal tables and external tables

Feature	Snowflake External Tables	Snowflake Internal Tables
Data storage location	Outside of Snowflake	Inside Snowflake
Data access method	Accessed via Snowflake external stage	Accessed using standard SQL statements
Data storage location	External storage system (e.g., S3, Azure Blob Storage, GCS)	Snowflake's internal storage system
Read/Write Operations	Read-only by default, but new data can be loaded using Snowpipe	Support both read and write operations
CREATE Statement	CREATE EXTERNAL TABLE	CREATE TABLE
Data Loading	Data is accessed directly from the external storage system	Data is accessed from Snowflake's internal storage
Data Ownership	Owned and managed by the external storage system	Owned and managed by Snowflake
Use cases	Storing data that is frequently accessed or updated	Storing data that is accessed less frequently or not updated

As external table is created using external stage first we should create storage integration then external stage after that w ecan create external table

Creating external table

CREATE OR REPLACE EXTERNAL TABLE ext_table

WITH LOCATION = @my_s3_stage

FILE_FORMAT = (TYPE = CSV);

```
--- AWS S3 Configuration.

create or replace storage integration s3_int
  type = external_stage
  storage_provider = s3
  enabled = true
  storage_aws_role_arn = 'arn:aws:iam::579834220952:role/snowflake_crc_role'
  storage_allowed_locations = ('s3://snowflakecrctest/emp/');

DESC INTEGRATION s3_int;

create or replace stage my_s3_stage
  storage_integration = s3_int
  url = 's3://snowflakecrctest/emp/'
  file_format = my_csv_format;

/************ External tables in snowflake ********************/

/************ Lecture: introduction to external tables ********************/

        CREATE OR REPLACE EXTERNAL TABLE ext_table
         WITH LOCATION = @my_s3_stage
         FILE_FORMAT = (TYPE = CSV);
```

As table will have only one column with variant data type .. the data is in json format. W ecan read data AS shown below.

```
select * from ext_table

select
value:c1::int as ID,
value:c2::varchar as name ,
value:c3::int as dept from sample_ext;
```

Create external table with column names

```
create or replace external table emp_ext_table
        (first_name string as  (value:c1::string),
        last_name string(20) as ( value:c2::string),
        email string as (value:c3::string))
        WITH LOCATION =  @my_s3_stage
         FILE_FORMAT = (TYPE = CSV);

        select * from emp_ext_table
```

External tables will store metadata

Whereas if we query data using only external stage .. in this scenario n snowflake don't maintain metadata. The main advantage of external table compare to querying directcyly from external stage is having metadata.

```
/*************** Lecture: Why external tables.(Get metadata information) *****************/

        select *
        from table(information_schema.external_table_files(TABLE_NAME=>'emp_ext_table'));

        56329893e1a6437c1224835c5197881d
        0d17bceb5358bb37c7766e999b1e9e3b

        select *
        from table(information_schema.external_table_file_registration_history(TABLE_NAME=>'emp_ext_table'));


        ALTER EXTERNAL TABLE emp_ext_table REFRESH;
```

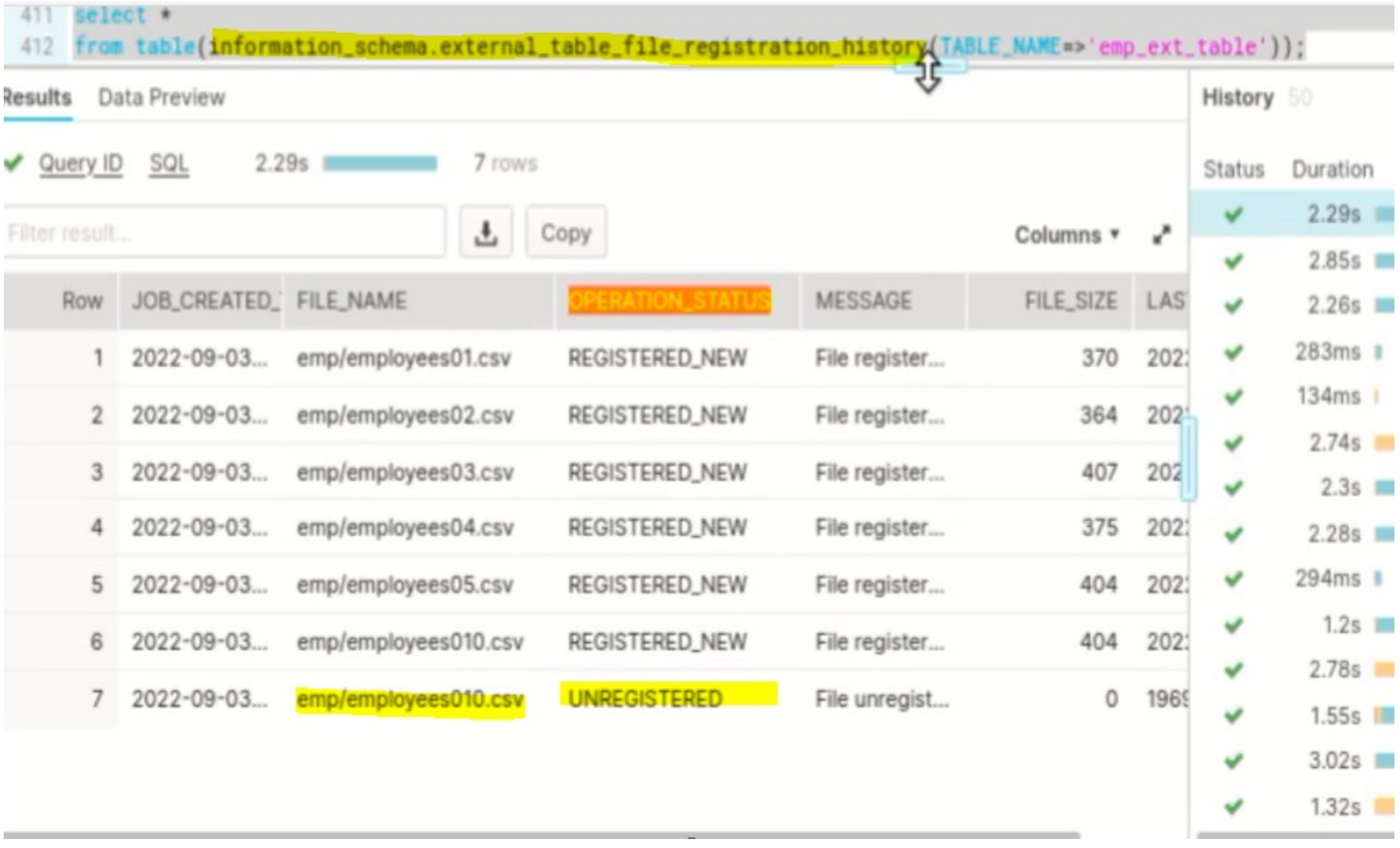
CASE 1: NEW FILES ADDED TO S3

If any new files added to s3 .. that data will not be reflected in external table.. to reflected that newly added files also we have to REFRESH external table so that metadata will be updated and we are able to see that newly added files data from external table

CASE 2: FILE DELETED FROM S3

If we delete the files from s3 .. but it wont reflect in external table as metadata is not updated . so we have to refresh metadata table then it will be reflected

From below image we can observe one file showing as unregistered .. which is deleted from s3.



Case 3: updating the s3 file then verify external table

As we updated s3 file.. and we did refresh external table…. Snowflake metadata will be updated..

The hash value for the updated file will change in external table so it will consider as file is updated so its showing as REGISTERED_UPDATE

Planning the schema of an external table

This section describes the options available for designing external tables.

Schema on read

All external tables include the following columns:

VALUE	A VARIANT type column that represents a single row in the external file.
METADATA$FILENAME	A pseudocolumn that identifies the name of each staged data file included in the external table, including its path in the stage.
METADATA$FILE_ROW_NUMBER	A pseudocolumn that shows the row number for each record in a staged data file.

To create external tables, you are only required to have some knowledge of the file format and record format of the source data files. Knowing the schema of the data files is not required.

Note that SELECT `*` always returns the VALUE column, in which all regular or semi-structured data is cast to variant rows.

PARTITIONS on external Tables

Performance Optimisation

As the data resides outside Snowflake, querying an external table may result in slower performance compared to querying an internal table stored within Snowflake. However, there are two ways to enhance the query performance for external tables. Firstly, when creating your External Table, you can improve performance by adding partitions. Alternatively, you can also create a Materialised View (Enterprise Edition Feature) based on the external table. Both approaches offer optimisations to expedite data retrieval and analysis from external tables.

Partitioning Your External Table

Partitioning your external tables is highly advisable, and to implement this, ensure that your underlying data is structured with logical paths incorporating elements like date, time, country or similar dimensions in the path. For this example, we will be partitioning news articles based on their published year. In this S3 bucket, I created three different folders, one for each year that will be used for partitioning. Each folder has a JSON file inside with multiple news articles for each year.

Partitions added automatically

An external table creator defines partition columns in a new external table as expressions that parse the path and/or filename information stored in the METADATA$FILENAME pseudocolumn. A partition consists of all data files that match the path and/or filename in the expression for the partition column.

The CREATE EXTERNAL TABLE syntax for adding partitions automatically based on expressions is as follows:

```
CREATE EXTERNAL TABLE
  <table_name>
     ( <part_col_name> <col_type> AS <part_expr> )
     [ , ... ]
  [ PARTITION BY ( <part_col_name> [, <part_col_name> ... ] ) ]
  ...
```

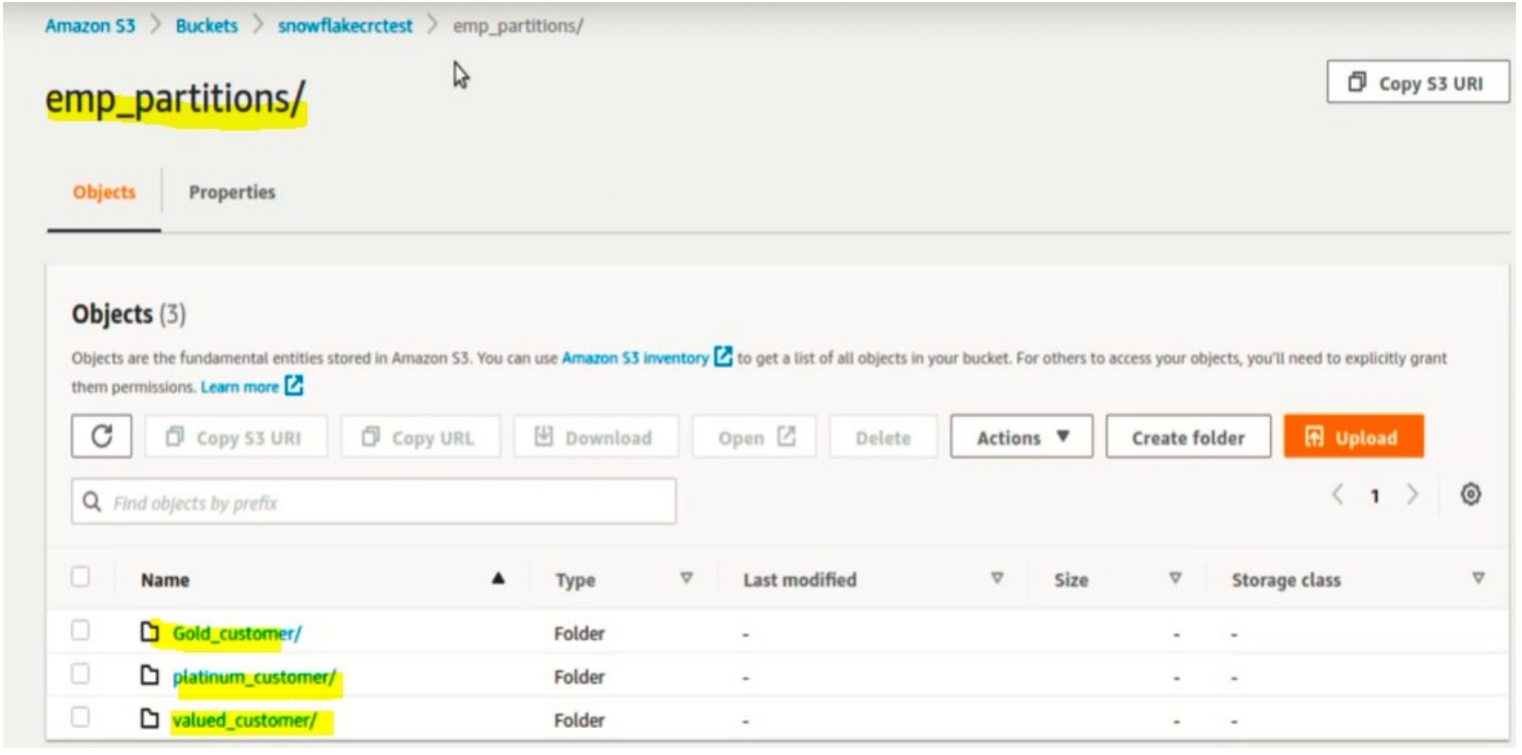
Snowflake computes and adds partitions based on the defined partition column expressions when an external table metadata is refreshed. By default, the metadata is refreshed automatically when the object is created. In addition, the object owner can configure the metadata to refresh automatically when new or updated data files are available in the external stage. The owner can alternatively refresh the metadata manually by executing the ALTER EXTERNAL TABLE ... REFRESH command.

```
create or replace external table emp_ext_table
        (
        file_name_part varchar AS SUBSTR(metadata$filename,5,11),
        first_name string as  (value:c1::string),
        last_name string(20) as ( value:c2::string),
        email string as (value:c3::string))
        PARTITION BY (file_name_part)
        WITH LOCATION =  @my_s3_stage
        FILE_FORMAT = (TYPE = CSV);

select * from emp_ext_table where file_name_part='employees03' and first_name in ('John','Chandru')
```

Manual partition in external tables (Add partitions manually)

. Use this option when you prefer to add and remove partitions selectively rather than automatically adding partitions for all new files in an external storage location that match an expression.

This option is generally chosen to synchronize external tables with other metastores (e.g. AWS Glue or Apache Hive).




```
create or replace external table emp_ext_table_partitions
        (
        customer_type string as  (parse_json(metadata$external_table_partition)::string),
        first_name string as  (value:c1::string),
        last_name string(20) as ( value:c2::string)
        )
        PARTITION BY (customer_type)
        PARTITION_TYPE = USER_SPECIFIED
        WITH LOCATION =  @my_s3_stage_partition
        FILE_FORMAT = (TYPE = CSV)
```

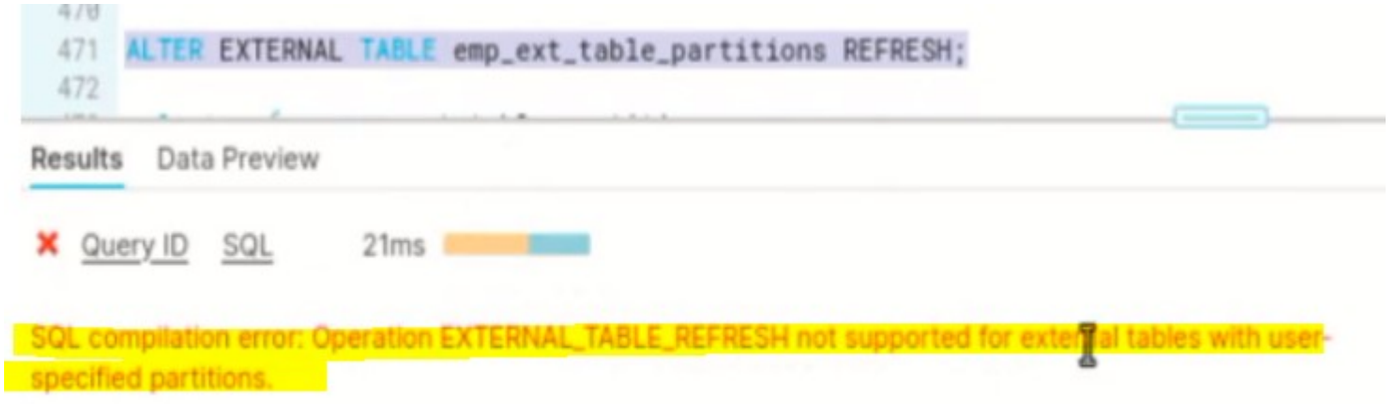
Include the required PARTITION_TYPE = USER_SPECIFIED parameter.

The partition column definitions are expressions that parse the column metadata in the internal (hidden) METADATA$EXTERNAL_TABLE_PARTITION column.

The object owner adds partitions to the external table metadata manually by executing the ALTER EXTERNAL TABLE … ADD PARTITION command:

```
ALTER EXTERNAL TABLE emp_ext_table_partitions ADD PARTITION (customer_type='Gold_customer')
location 'Gold_customer/'    ;

ALTER EXTERNAL TABLE emp_ext_table_partitions ADD PARTITION (customer_type='platinum_customer')
location 'platinum_customer/'    ;


select * from emp_ext_table_partitions

select * from emp_ext_table_partitions where customer_type='Gold_customer';

ALTER EXTERNAL TABLE emp_ext_table_partitions REFRESH;--it work for manual added partitions
```

Note: External table REREFSH wont work for manual adding partions ..it will throw error



Refreshing external table metadata automatically

```
create or replace external table emp_ext_table
        (
        file_name_part varchar AS SUBSTR(metadata$filename,5,11),
        first_name string as  (value:c1::string),
        last_name string(20) as ( value:c2::string),
        email string as (value:c3::string))
        PARTITION BY (file_name_part)
        WITH LOCATION =  @my_s3_stage
        FILE_FORMAT = (TYPE = CSV)
        auto_refresh=true;
```

Auto refresh external tables

- Using SQS queue.
- Using SNS topic..

Step 3: Configure event notifications

Configure event notifications for your S3 bucket to notify Snowflake when new or updated data is available to read into the external table metadata. The auto-refresh feature relies on SQS queues to deliver event notifications from S3 to Snowflake.

For ease of use, these SQS queues are created and managed by Snowflake. The SHOW EXTERNAL TABLES command output displays the Amazon Resource Name (ARN) of your SQS queue.

1. Execute the SHOW EXTERNAL TABLES command:

```
SHOW EXTERNAL TABLES;
```

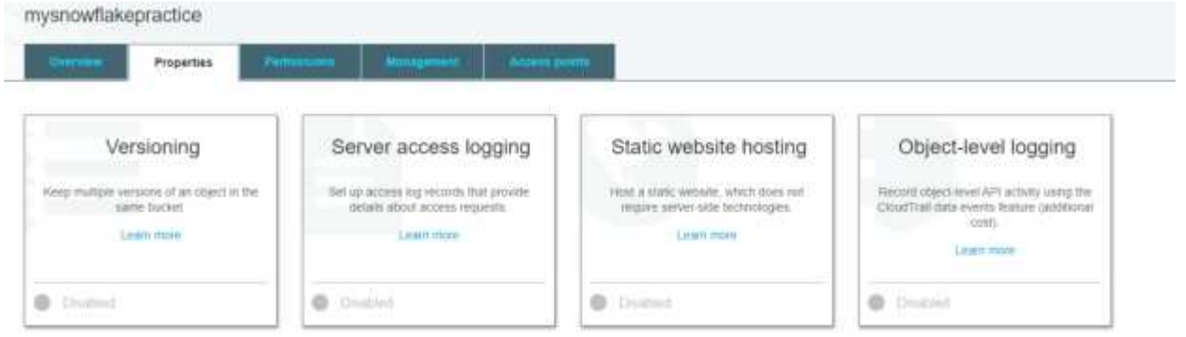
Note the ARN of the SQS queue for the external table in the `notification_channel` column. Copy the ARN to a convenient location.

> **Note**
> Following AWS guidelines, Snowflake designates no more than one SQS queue per S3 bucket. This SQS queue may be shared among multiple buckets in the same AWS account. The SQS queue coordinates notifications for all external tables reading data files from the same S3 bucket. When a new or modified data file is uploaded into the bucket, all external table definitions that match the stage directory path read the file details into their metadata.

2. Log into the AWS Management Console.
3. Configure an event notification for your S3 bucket using the instructions provided in the Amazon S3 documentation. Complete the fields as follows:
   - **Name**: Name of the event notification (e.g. `Auto-ingest Snowflake`).
   - **Events**: Select the **ObjectCreate (All)** and **ObjectRemoved** options.
   - **Send to**: Select **SQS Queue** from the dropdown list.
   - **SQS**: Select **Add SQS queue ARN** from the dropdown list.
   - **SQS queue ARN**: Paste the SQS queue name from the SHOW EXTERNAL TABLES output.

To create AWS notification :

Go to properties→ under advance settings →select events→click on add notifications→



Enter details as shown below

Here main noting point is we have to select SQS queue

In sqs ARN value we have to copy key from snow flake show pipe notification key value.

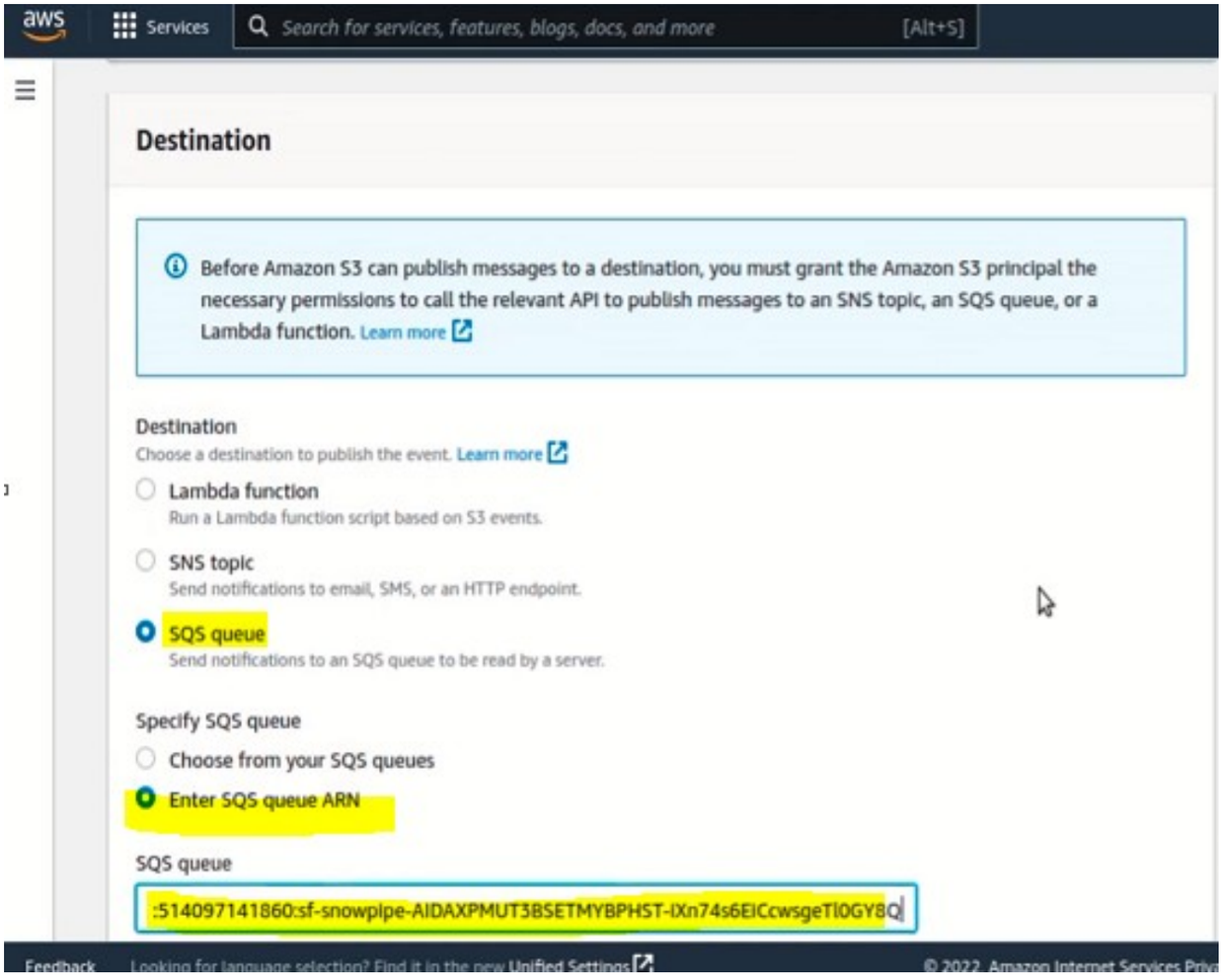
Object creation

- [ ] **All object create events**
  s3:ObjectCreated:*
- [ ] **Put**
  s3:ObjectCreated:Put
- [ ] **Post**
  s3:ObjectCreated:Post
- [ ] **Copy**
  s3:ObjectCreated:Copy
- [ ] **Multipart upload completed**
  s3:ObjectCreated:CompleteMultipartUpload

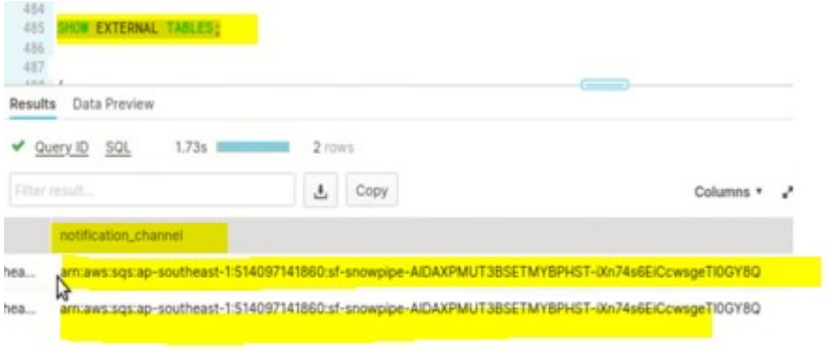
Object removal

- [ ] **All object removal events**
  s3:ObjectRemoved:*
- [ ] **Permanently deleted**
  s3:ObjectRemoved:Delete
- [ ] **Delete marker created**
  s3:ObjectRemoved:DeleteMarkerCreated

Object restore



Sqs queue ARN value has to be taken from snow flake **show external tables** notification_channel column value



Refreshing Your External Table Metadata

When creating your external table, you can set the parameter `AUTO_REFRESH` to `TRUE`. This automatically refreshes the metadata when new or updated data files are available in the stage you used when creating your external table. For example:

```
CREATE OR REPLACE EXTERNAL TABLE RANDOMLY_GENERATED_JSON
WITH LOCATION = @JSON_STAGE/
FILE_FORMAT = MY_JSON_FILE_FORMAT
AUTO_REFRESH = TRUE
;
```

It's important to note that setting `AUTO_REFRESH` to `TRUE` is not supported if you created your external table with manually added partitions (for example, when `PARTITION_TYPE = USER_SPECIFIED`). When an external table is created, its metadata is refreshed automatically once unless `REFRESH_ON_CREATE = FALSE`. To ensure that Snowflake is notified when new or updated data becomes available for reading into the external table metadata, it is essential to set up an event notification for your storage location. You can learn more about how to set up these event notifications in the following resources:

- For Event Notifications in AWS: Automated Ingestion from AWS S3 into Snowflake via Snowpipe.
- For Event Notifications in Azure: Automated Ingestion from Azure Storage into Snowflake via Snowpipe.

Materialized Views (Enterprise Edition Feature)

A materialized view is an optimised and pre-computed data set that stems from a query specification (the SELECT statement in the view definition) and is stored for future use. By pre-computing the data, querying a materialized view is considerably faster compared to executing the same query against the base table from which the view is derived. This performance advantage becomes especially significant when dealing with complex and frequently executed queries.

Materialized views excel at speeding up resource-intensive operations such as aggregation, projection and selection, particularly when these operations are performed regularly on vast datasets. Their ability to store and present pre-processed data allows for quicker access and minimises the need for repeated, computationally expensive calculations.

The example above works well for simple JSON structures; however, it can become cumbersome when your JSON objects have nested elements. In cases like this, using a materialized view to flatten and parse the semi-structured data into a structured, tabular format for users to query can be extremely beneficial. For example, imagine a file with the randomly generated JSON below:

```
1.  CREATE OR REPLACE EXTERNAL TABLE RANDOMLY_GENERATED_JSON
2.  WITH LOCATION = @MY_STAGE/users
3.  FILE_FORMAT = MY_JSON_FILE_FORMAT
4.  ;
```

Once we have this data loaded in our external table, we can create the Materialized View to flatten and parse the data:

```
CREATE OR REPLACE MATERIALIZED VIEW USERS_JSON AS
SELECT
    $1:profile.name AS NAME
    ,$1:email AS EMAIL
    ,$1:username AS USERNAME
    ,$1:profile.company AS COMPANY
    ,$1:profile.dob AS DATE_OF_BIRTH
    ,$1:profile.address AS ADDRESS
    ,$1:profile.location.lat AS LATITUDE
    ,$1:profile.location.long AS LONGITUDE
    ,$1:roles AS ROLES FROM RANDOMLY_GENERATED_JSON
;
```

This example serves as a simplified representation of semi-structured data handling. In reality, real-world semi-structured datasets can be remarkably more intricate and deeply nested.

ICEBERG TABLES

Refer ch Pradeep udemy snowflake videos --- this videos covers iceberg tables with snowflake has catalog ie. Read/write works

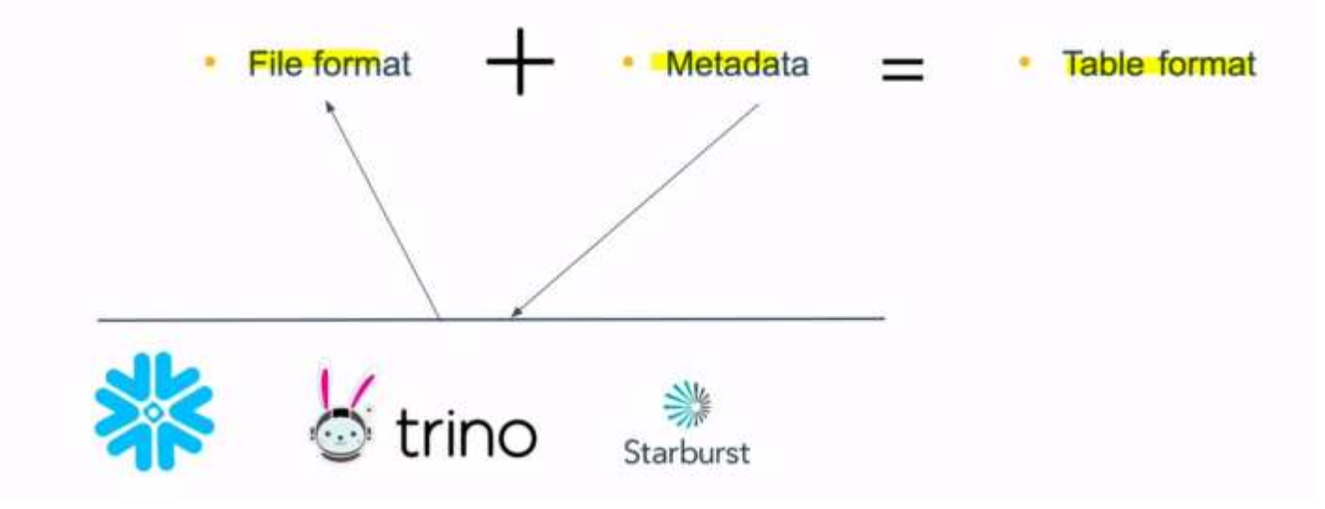
https://www.snowflake.com/blog/iceberg-tables-powering-open-standards-with-snowflake-innovations/

https://www.phdata.io/blog/what-are-iceberg-tables-in-snowflake-and-when-to-use-them/

https://www.snowflake.com/blog/5-reasons-apache-iceberg/

What is Apache Iceberg?

- **Apache Iceberg is an open table format for huge analytic datasets.** Iceberg adds tables to compute engines including Spark,Trino,PrestoDB, Flink, Hive and Impala using a high-performance table format that works just like a SQL table.
- Why we need Table format ?
- individual files in a data lake don't contain enough information needed by query engines and other applications for effective pruning, time travel, schema evolution
- Table formats solve these issues by providing metadata.



Apache Iceberg is a high-performance open table format designed to manage huge datasets at scale. This format determines how to manage, organize, and track all the data files stored in open file formats that make up a table.

What is Apache Iceberg?

Iceberg is easiest to explain with a thought experiment. Imagine you have a thousand Parquet files in a cloud storage bucket. If you have three different readers of that data, they all need to know which files correspond to a table. There could be one table that points to all of the files, or many tables pointing to a subset of the files. These users may also want to know how the table has changed over time, or optimize queries on the table. Table formats like Iceberg are designed to solve this problem—they provide rich table metadata on top of files.

While there are several table formats to choose from, we have chosen to support Iceberg because we believe Iceberg represents the best next-generation table format. Iceberg is really exciting to us because it solves some tricky problems for customers and users, including:

- **Engine agnostic:** Iceberg was designed from inception to be engine agnostic. This means you get rich functionality across several analytical engines, and critical functionality is not tied to only one engine.
- **Diversity of thought:** The Iceberg project sees active participation *and* contribution from many different organizations, and is not tied to any commercial entity. This means the project benefits from ideas from many different viewpoints and customer needs. Moreover, with Iceberg there's no concern about who is controlling the project and to what ends.
- **Rich ecosystem:** Many different open source projects have adopted support for or extended Apache Iceberg. Moreover, commercial support for first-class Iceberg has been growing. This means customers have more capabilities and options with Iceberg.

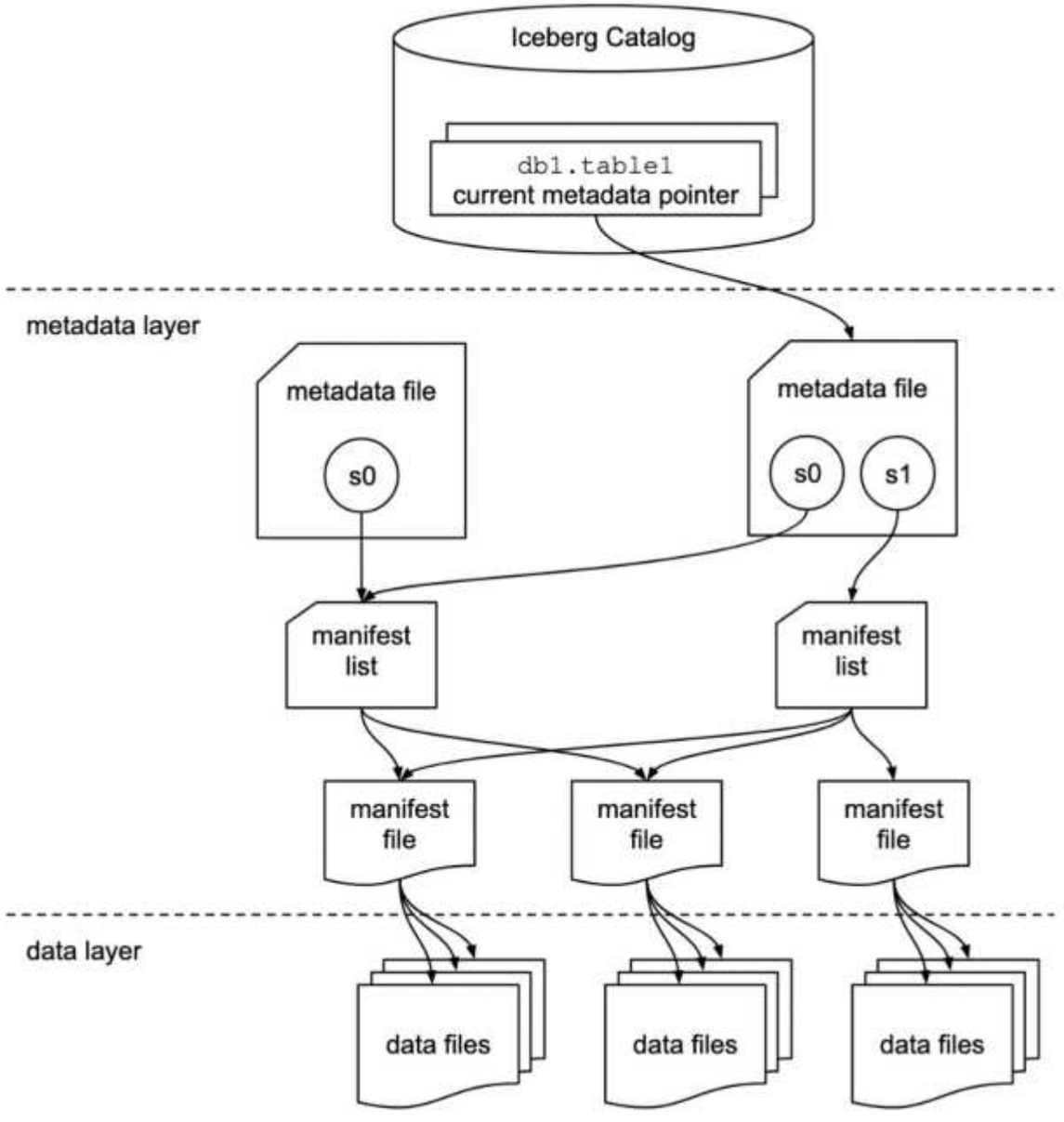
For more detail on reasons to consider Iceberg, read this blog post. And if you are interested in Iceberg, we encourage you to check out the Iceberg Community.

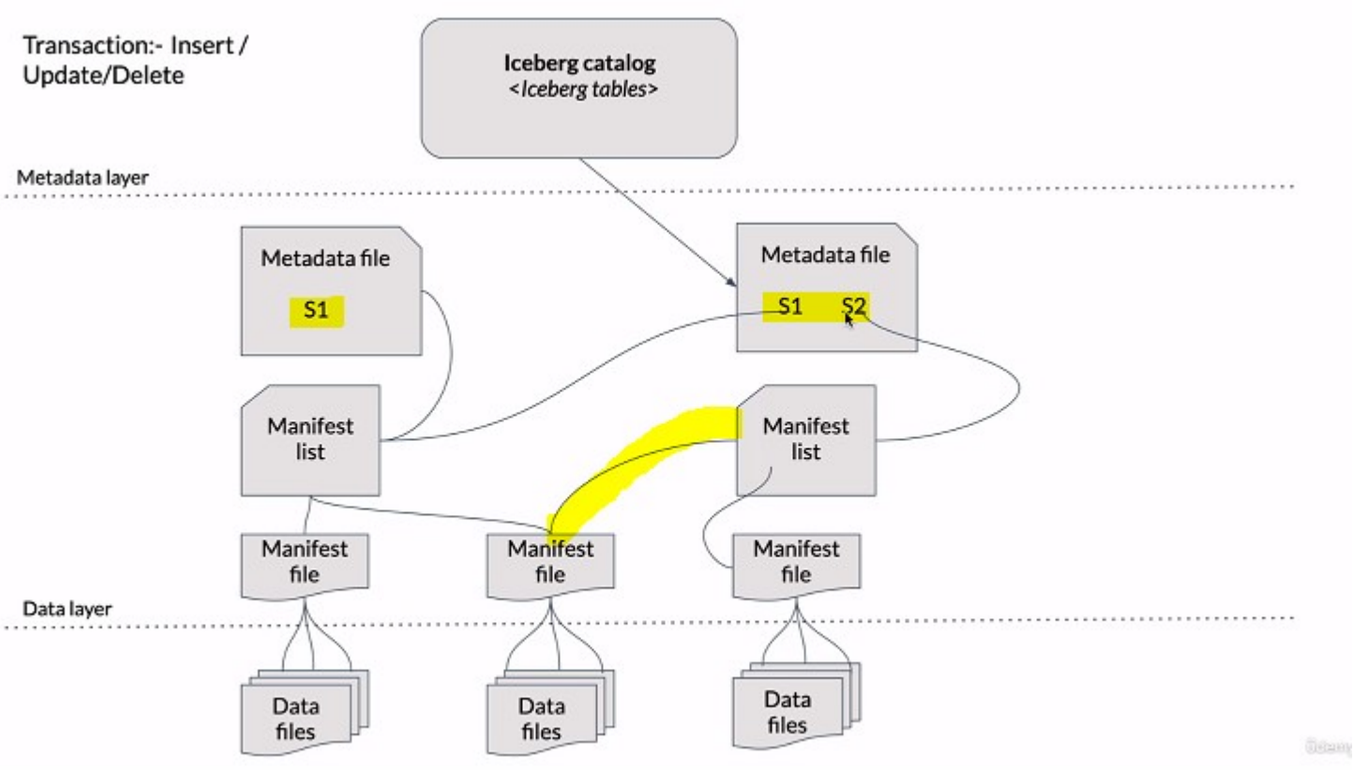
Mail

Architecture Overview

The architecture of Apache Iceberg consists of three different layers:

- Catalog
- Metadata layer
- Data layer


Iceberg Catalog
db1.table1
current metadata pointer
metadata layer
metadata file
s0
metadata file
s0
s1
manifest list
manifest list
manifest file
manifest file
manifest file
data layer
data files
data files
data files


Transaction:- Insert / Update/Delete
Iceberg catalog
<Iceberg tables>
Metadata layer
Metadata file
S1
Metadata file
S1
S2
Manifest list
Manifest list
Manifest file
Manifest file
Manifest file
Data layer
Data files
Data files
Data files

Catalog

The catalog manages a collection of tables and tracks the table's current metadata. Additionally, it supports atomic operations to update current metadata pointers. Hive metastore, AWS Glue, Nessie, JDBC database, Snowflake, etc., can all be used as catalogs for Iceberg tables.

Metadata File

The metadata file stores the metadata of a table at a given timestamp in JSON format. This includes details about table snapshots, manifest lists, schema, partition spec, etc. Every time there is a change in table data or metadata, a new metadata file is created.

Manifest List

The manifest list is a group of manifest files that are linked to a snapshot. These are normally avro files that store details about manifest files, including statistics.

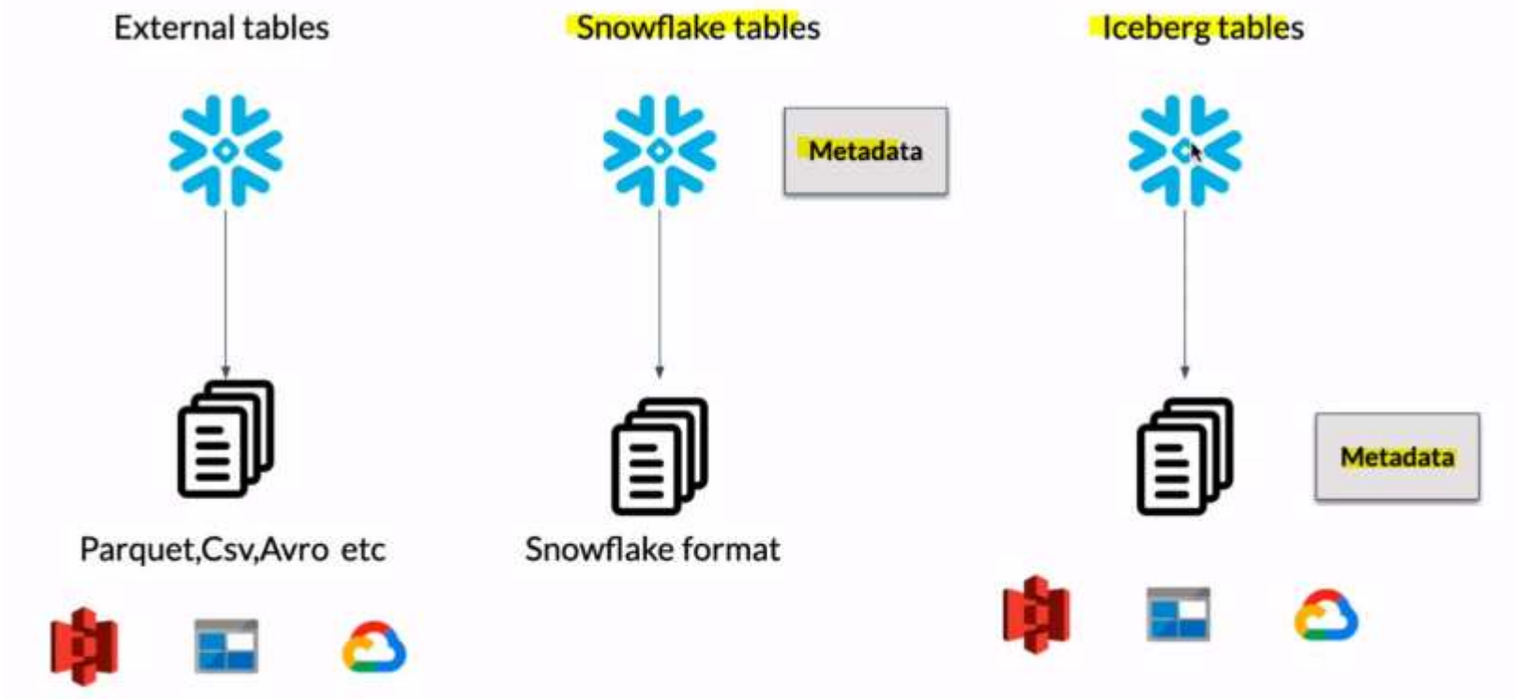
Manifest File

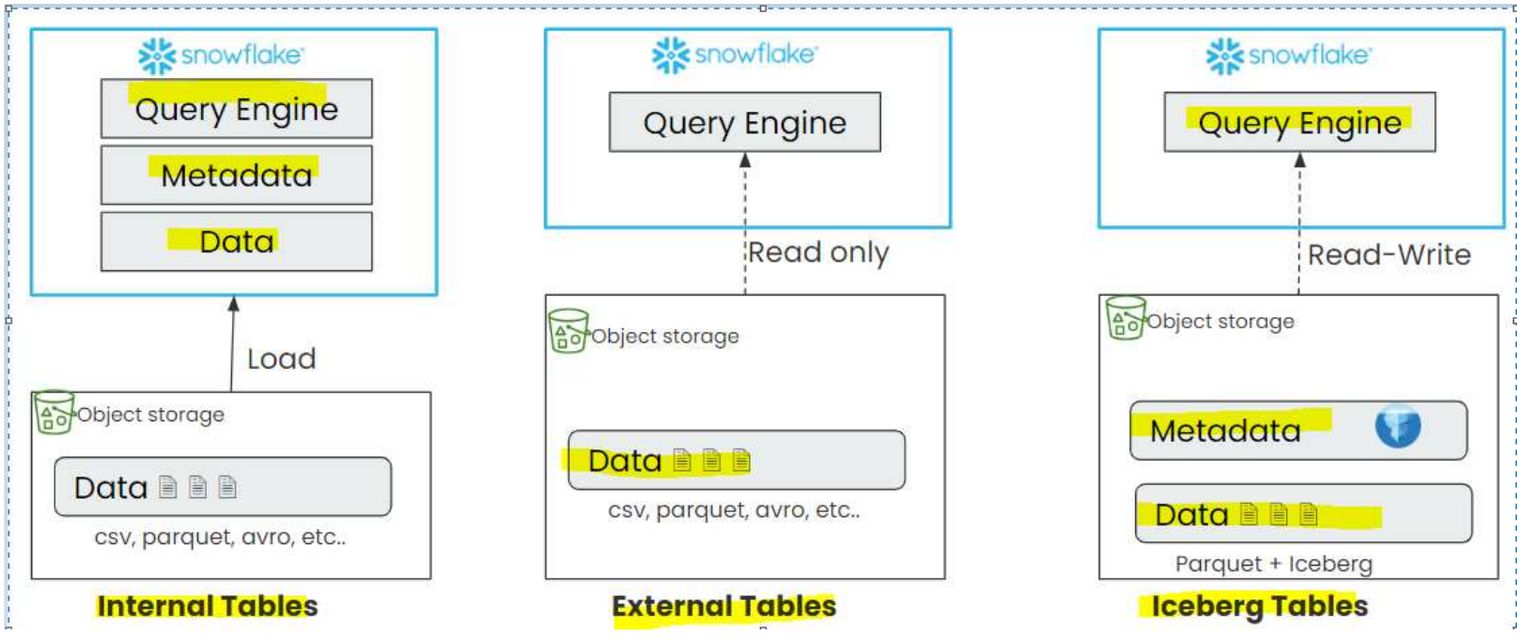
This is also an Avro file, which contains a list of data files with column statistics and partition information for each data file.

Data Files

Data files are the actual files storing data in open file formats: ORC, parquet, and AVRO.

COMPARIISON OF external tables vs snowflake tables vs iceberg table





Key Differences Between Snowflake Native Tables and Iceberg Tables

1. Table metadata is stored in Iceberg format in public cloud storage and optionally in Snowflake if Snowflake is used as a catalog.
2. Data is stored in parquet format in public cloud storage, not in Snowflake.
3. Both Snowflake and external compute engines like Spark can read data from cloud storage and write back to the same location.

Configure iceburg with snowflake

- s3 bucket
- IAM policy
- IAM role
- Add trust relationship

Data storage

Iceberg tables store their data and metadata files in an external cloud storage location (Amazon S3, Google Cloud Storage, or Azure Storage). The external storage is not part of Snowflake. You are responsible for all management of the external cloud storage location, including the configuration of data protection and recovery. Snowflake does not provide Fail-safe storage for Iceberg tables.

Snowflake connects to your storage location using an external volume, and Iceberg tables incur no Snowflake storage costs. For more information, see Billing.

To learn more about storage for Iceberg tables, see Iceberg table storage.

External volume

An external volume is a named, account-level Snowflake object that you use to connect Snowflake to your external cloud storage for Iceberg tables. An external volume stores an identity and access management (IAM) entity for your storage location. Snowflake uses the IAM entity to securely connect to your storage for accessing table data, Iceberg metadata, and manifest files that store the table schema, partitions, and other metadata.

A single external volume can support one or more Iceberg tables.

To set up an external volume for Iceberg tables, see Configure an external volume for Iceberg tables.

Iceberg catalog

An Iceberg catalog enables a compute engine to manage and load Iceberg tables. The catalog forms the first architectural layer in the Iceberg table specification and must support:

- Storing the current metadata pointer for one or more Iceberg tables. A metadata pointer maps a table name to the location of that table's current metadata file.
- Performing atomic operations so that you can update the current metadata pointer for a table.

To learn more about Iceberg catalogs, see the Apache Iceberg documentation.

Snowflake supports different catalog options. For example, you can use Snowflake as the Iceberg catalog, or use a catalog integration to connect Snowflake to an external Iceberg catalog.

Catalog integration

A catalog integration is a named, account-level Snowflake object that stores information about how your table metadata is organized when you don't use Snowflake as the Iceberg catalog. For example, you need a catalog integration if your table is managed by AWS Glue.

A single catalog integration can support one or more Iceberg tables that use the same external catalog.

To set up a catalog integration, see Configure a catalog integration for Iceberg tables.

Metadata and snapshots

Iceberg uses a snapshot-based querying model, where data files are mapped using manifest and metadata files. A snapshot represents the state of a table at a point in time and is used to access the complete set of data files in the table.

To learn about table metadata and Time Travel support, see Metadata and retention.

Iceberg Table Types in Snowflake : **-----IMP FROM DOCUMENTATION**

1. **Snowflake as the Iceberg catalog**
2. **Use an external catalog**

Iceberg catalog options

When you create an Iceberg table in Snowflake, you can use Snowflake as the Iceberg catalog or you can use a catalog integration to connect to an external Iceberg catalog.

The following table summarizes the differences between these catalog options.

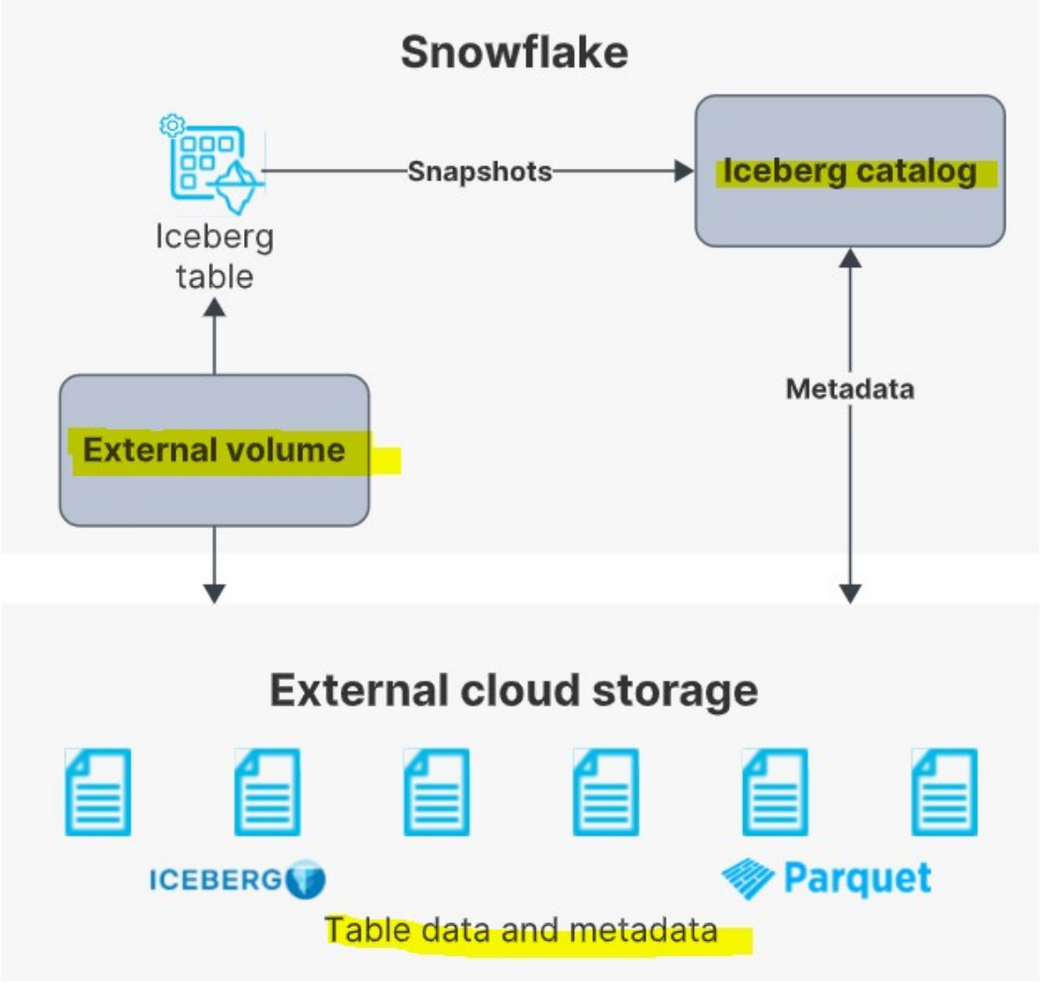
	Use Snowflake as the Iceberg catalog	Use an external catalog
Read access	✓	✓
Write access	✓	✗ For Snowflake platform support, you can convert the table to use Snowflake as the catalog.
Data and metadata storage	External volume (cloud storage)	External volume (cloud storage)
Snowflake platform support	✓	✗
Works with the Snowflake Iceberg Catalog SDK	✓	✓

IMP: ICEBERG TABLE TYPE COMPARISION From snowflake documentation

	Use Snowflake as the Iceberg catalog	Use an external catalog
Read access	✓	✓
Write access	✓	✗ For Snowflake platform support, you can convert the table to use Snowflake as the catalog.
Data and metadata storage	External volume (cloud storage)	External volume (cloud storage)
Snowflake platform support	✓	✗
Works with the Snowflake Iceberg Catalog SDK	✓	✓
Cross-cloud/cross-region tables	✗ wont support Cross-cloud/cross-region support	✓
clustering	✓	✗

Use Snowflake as the catalog

An Iceberg table that uses Snowflake as the Iceberg catalog provides full Snowflake platform support with read and write access. The table data and metadata are stored in external cloud storage, which Snowflake accesses using an **external volume.** Snowflake handles all life-cycle maintenance, such as compaction, for the table.



Use an external catalog

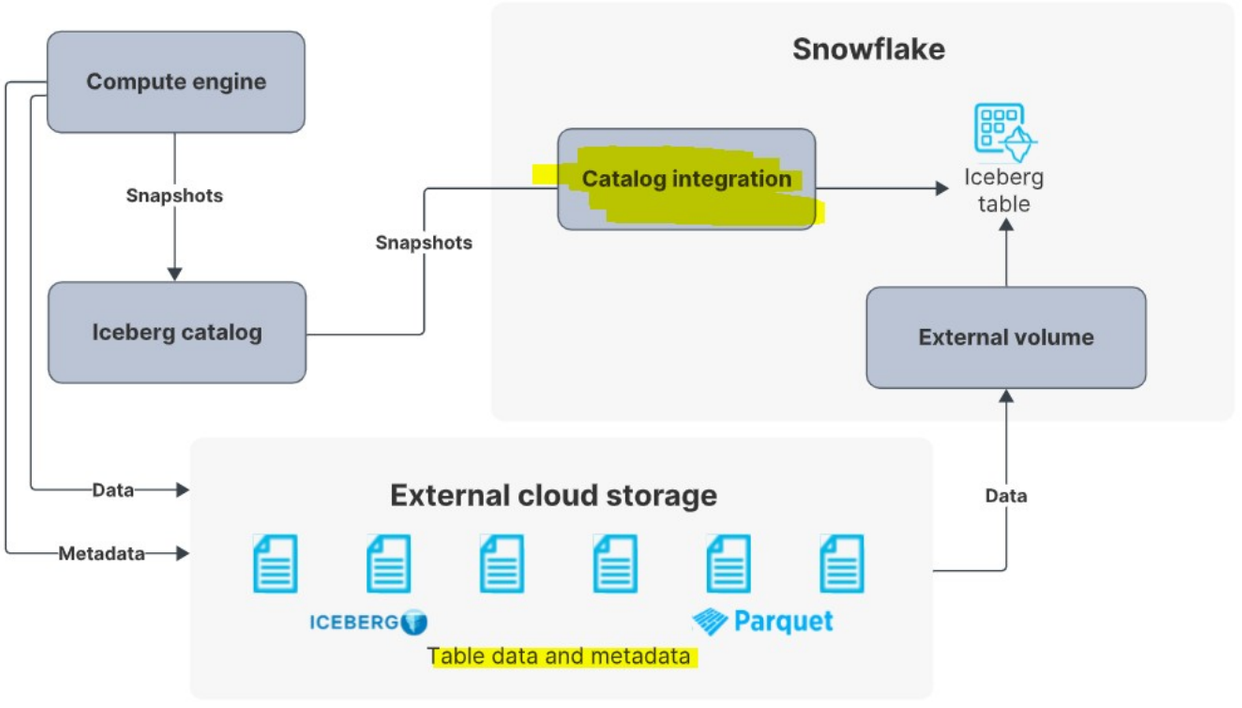
An Iceberg table that uses an external catalog provides limited Snowflake platform support with read-only access. With this table type, Snowflake uses a **catalog integration** to retrieve information about your Iceberg metadata and schema.

You can use this option to create an Iceberg table registered in the AWS Glue Data Catalog or to create a table from Iceberg metadata files in object storage.

Snowflake does not assume any life-cycle management on the table.

The table data and metadata are stored in external cloud storage, which Snowflake accesses using an external volume.

The following diagram shows how an Iceberg table uses a catalog integration with an external Iceberg catalog.



Catalog integration

A catalog integration is a named, account-level Snowflake object that stores information about how your table metadata is organized when you don't use Snowflake as the Iceberg catalog. For example, you need a catalog integration if your table is managed by AWS Glue.

A single catalog integration can support one or more Iceberg tables that use the same external catalog.

To set up a catalog integration, see Configure a catalog integration for Iceberg tables.

Iceberg Table Types in Snowflake

https://www.phdata.io/blog/what-are-iceberg-tables-in-snowflake-and-when-to-use-them/

Depending on where the catalog is managed for Iceberg tables, Snowflake can have two types of Iceberg tables:

1. **Snowflake Managed Iceberg Tables** – Snowflake manages the metadata and catalog for these tables. These tables can support all Snowflake features with read and write access.

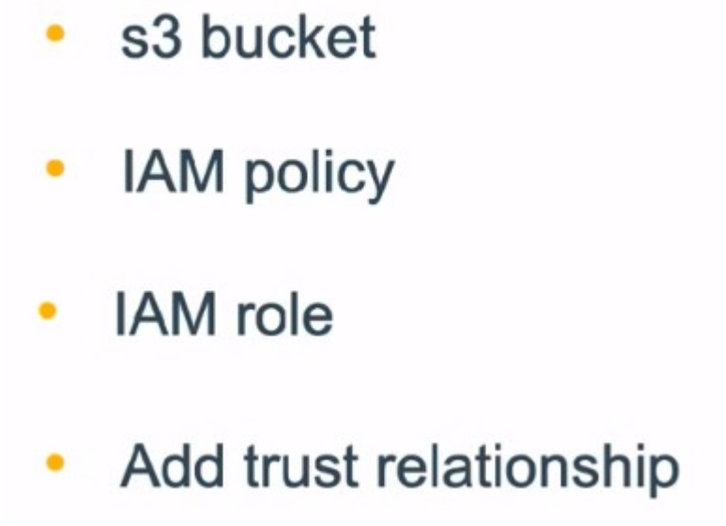
2. **Externally Managed Iceberg Tables** – An external system such as AWS Glue manages the metadata and catalog. These tables can support read-only access in Snowflake.

Feature	Snowflake Managed Iceberg tables	Externally Managed Iceberg tables
Read access from Snowflake	✔	✔
Write access from Snowflake	✔	✘
Use of warehouse cache for queries	✔	✔
Automatic metadata refresh in Snowflake	✔	✘
Snowflake platform features: time travel, row/column mask, etc.	All features	Limited features
Nested datatype support	✔	✔
Support of table clustering	✔	✘

Note: Tables that use Snowflake as the catalog wont support Cross-cloud/cross-region support

Cross-cloud/cross-region tables are supported when you use an external catalog

Hands on iceberg tables(from Pradeep video) catalog managed by snow flake



1. cretate external volume

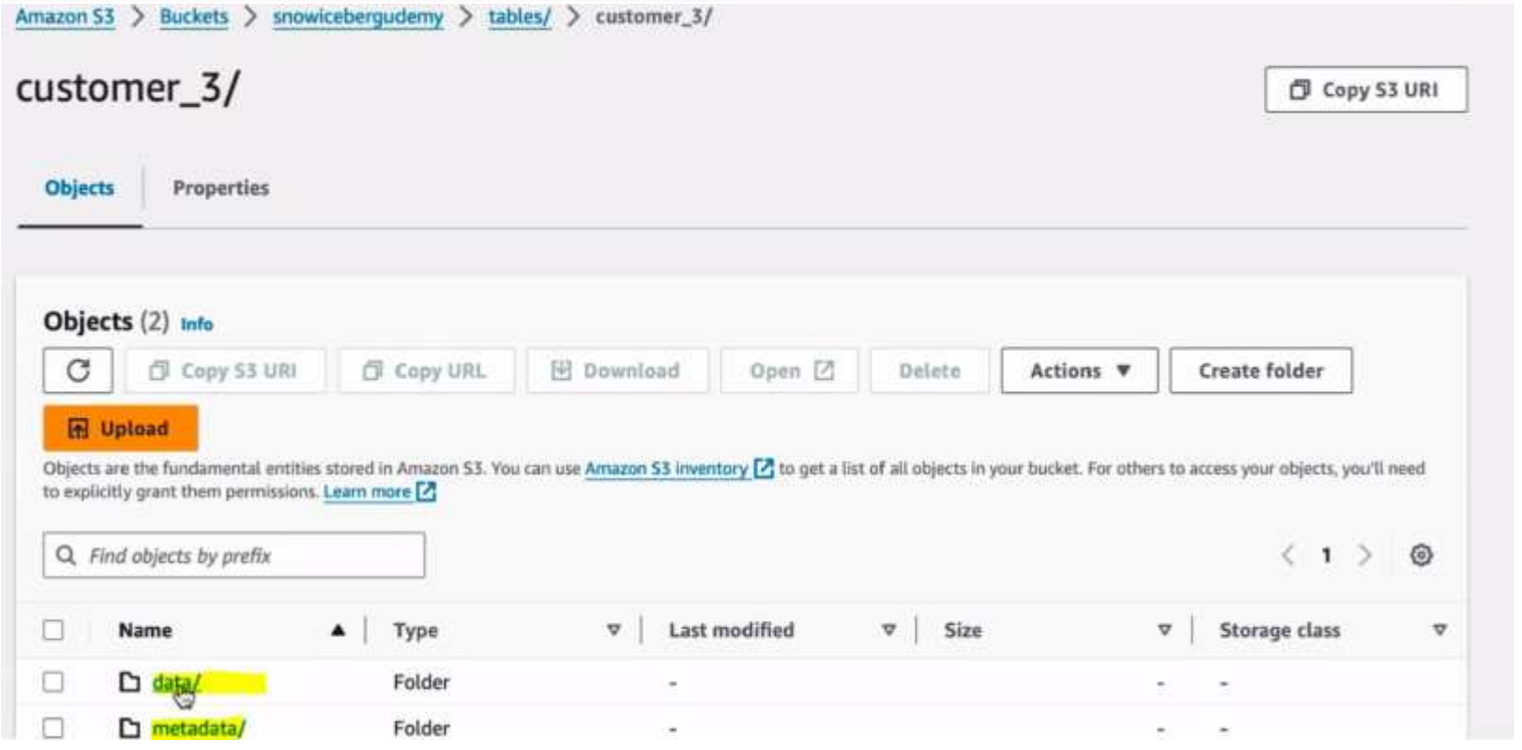
```
CREATE OR REPLACE EXTERNAL VOLUME exvol
   STORAGE_LOCATIONS =
      (
         (
            NAME = 'iceberg_test'
            STORAGE_PROVIDER = 'S3'
            STORAGE_BASE_URL = 's3://snowicebergudemy/tables/'
            STORAGE_AWS_ROLE_ARN = 'arn:aws:iam::579834220952:role/snowiceberg_role'

         )
      );

      DESC EXTERNAL VOLUME exvol;
```

3.create table

```
CREATE OR REPLACE ICEBERG TABLE customer_iceberg (
    c_custkey INTEGER,
    c_name STRING,
    c_address STRING,
    c_nationkey INTEGER,
    c_phone STRING,
    c_acctbal INTEGER,
    c_mktsegment STRING,
    c_comment STRING
)
    CATALOG='SNOWFLAKE'
    EXTERNAL_VOLUME='exvol'
    BASE_LOCATION='';
```

```
BASE_LOCATION  points to this s3  's3://snowicebergudemy/tables/'
```

```
INSERT INTO customer_iceberg
  SELECT * FROM snowflake_sample_data.tpch_sf1.customer where c_acctbal='9957.56'
```



Query and Time Travel

Iceberg Tables are treated much like other tables in Snowflake. For example, you can read different table types in a single query. This query is joining an Iceberg Table with a traditional Snowflake Table.

```
SELECT
    *
FROM customer_iceberg c
INNER JOIN snowflake_sample_data.tpch_sf1.nation n
    ON c.c_nationkey = n.n_nationkey;
```

Benefits of the additional metadata that table formats like Iceberg and Snowflake's provide are, for example, time travel.

Let's first make a simple update to the table. Then, you can see that the row count has increased compared to the previous version of the table.

```
INSERT INTO customer_iceberg
    SELECT
        *
    FROM snowflake_sample_data.tpch_sf1.customer
    LIMIT 5;


SELECT
    count(*) AS after_row_count,
    before_row_count
FROM customer_iceberg
JOIN (
        SELECT count(*) AS before_row_count
        FROM customer_iceberg BEFORE(statement => LAST_QUERY_ID())
    )
    ON 1=1
GROUP BY 2;
```

COPY ON WRITE (COW)

- There are two approaches to handle deletes and updates in the data lakehouse: copy-on-write (COW) and merge-on-read (MOR).
- **Copy-on-write (COW) is the default mode for writing Iceberg tables in Snowflake.**
- Copy-On-Write (COW) – **Best for tables with frequent reads**, infrequent writes/updates, or large batch updates
- Merge-On-Read (MOR) – **Best for tables with frequent writes/updates**

Copy-On-Write (COW) – Best for tables with frequent reads, infrequent writes/updates, or large batch updates

With COW, when a change is made to delete or update a particular row or rows, the datafiles with those rows are duplicated, but the new version has the updated rows. This makes writes slower depending on how many data files must be re-written which can lead to concurrent writes having conflicts and potentially exceeding the number of reattempts and failing.

If updating a large number of rows, COW is ideal. However, if updating just a few rows you still have to rewrite the entire data file, making small or frequent changes expensive.

On the read side, COW is ideal as there is no additional data processing needed for reads – the read query has nice big files to read with high throughput.

Copy on Write

Summary of COW (Copy-On-Write)	
PROS	CONS
Fastest reads	Expensive writes
Good for infrequent updates/deletes	Bad for frequent updates/deletes

Merge-On-Read (MOR) – Best for tables with frequent writes/updates

With merge-on-read, the file is not rewritten, instead the changes are written to a new file. Then when the data is read, the changes are applied or merged to the original data file to form the new state of the data during processing.

This makes writing the changes much quicker, but also means more work must be done when the data is read.

In Apache Iceberg tables, this pattern is implemented through the use of delete files that track updates to existing data files.

If you delete a row, it gets added to a delete file and reconciled on each subsequent read till the files undergo compaction which will rewrite all the data into new files that won't require the need for the delete file.

If you update a row, that row is tracked via a delete file so future reads ignore it from the old data file and the updated row is added to a new data file. Again, once compaction is run, all the data will be in fewer data files and the delete files will no longer be needed.

So when a query is underway the changes listed in the delete files will be applied to the appropriate data files before executing the query.

SNOWFLAKE HYBRID TABLES—IN PREVIEW jun 2024

https://medium.com/snowflake/snowflake-hybrid-tables-all-you-need-to-know-58fd73426698

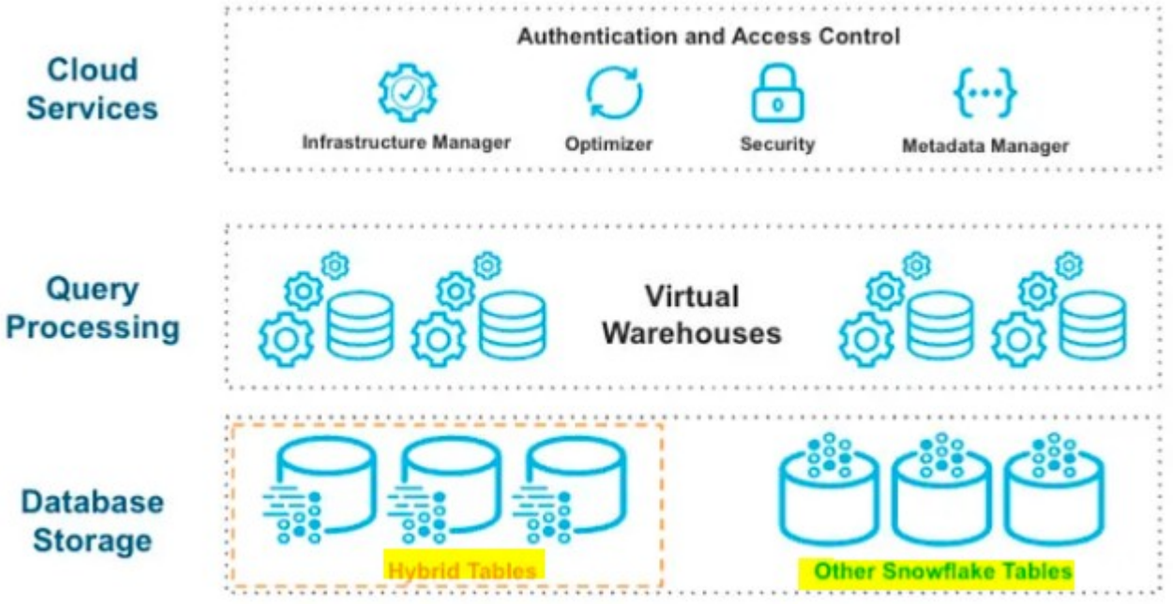
it supports indexing, unique constraints and integrtity checks like oracle

Snowflake has released the Hybrid Tables feature into Public Preview, which has changed how things work. They can now perform OLAP (Online Analytical Processing) and OLTP (Online Transaction Processing), which means that a single table type can deliver fast transaction-based performance. As someone with inside knowledge, I have seen how performance has improved since it was first announced in a private preview. As we move into Public Preview, there have been notable improvements in analytical workloads by 50X and transactional workloads by 10X.

Hybrid Tables Overview

Hybrid tables give you fast, high-concurrency point operations with unique constraints, indexing, and integrity checks.

It is a different type of table compared to the Snowflake Traditional Standard tables, and the infrastructure behind it is unique. The secret sauce is the combination of columnar key-value pair databases and caching. Like Iceberg tables, it is just a distinct implementation of standard tables.



One of the most significant differences between HYBRID and Standard Tables is that HYBRID uses row locking, allowing for much higher concurrency and throughput.

Unique features of HYBRID Tables:

- Primary keys are unique identifiers within a table and enable fast analysis. Hybrid Tables consistently enforce primary keys, while other Snowflake tables provide primary keys but do not enforce them.
- Foreign keys create a connection between data in different tables by referring to primary keys.
- Referential integrity validates the data before loading to implement foreign key relationships and prevent destructive actions.
- Secondary indexes are utilized to speed up the querying of data based on an attribute other than the primary key. This super simple process creates an index on a non-primary vital column.
- Hybrid Tables can be queried and joined with any other table in Snowflake.
- Row locking instead of table locking for writes
- All indexing and maintenance will continue to run behind the scenes without the need to be managed by the customer

Feature	Hybrid tables	Standard tables
Primary data layout	Row oriented, with secondary columnarization	Columnar micro-partitions
Locking	Row locking	Partition or table locking
Primary keys	Required; enforced uniqueness	Optional, not enforced
Foreign keys	Optional, enforced referential integrity	Optional, not enforced
Constraints	Supports enforcement of unique constraints, referential integrity constraints	Not supported
Indexes	Supported for performance; indexes are updated synchronously on write	Search Optimization indexes columns for better point lookup performance; batch updated/maintained asynchronously

How To Use It

Like any other table, it is easy to launch using the following SQL command.

```
-- Create hybrid table
CREATE OR REPLACE HYBRID TABLE augusto_hybrid (
  id NUMBER PRIMARY KEY AUTOINCREMENT START 1 INCREMENT 1,
  col1 VARCHAR NOT NULL,
  col2 VARCHAR NOT NULL
);
```

But let’s look at a case with FOREIGN KEY and Secondary Indexes

```
CREATE OR REPLACE HYBRID TABLE Orders (
    Orderkey number(38,0) PRIMARY KEY,
    Customerkey number(38,0),
    Orderstatus varchar(20),
    Totalprice number(38,0),
    Orderdate timestamp_ntz,
    Clerk varchar(50),
CONSTRAINT fk_o_customerkey FOREIGN KEY (Customerkey) REFERENCES Customers(Customerkey),
INDEX index_o_orderdate (Orderdate)); -- secondary index to accelerate time-based lookups
```

Note that analytical queries inside HYBRID Tables are around 2 to 3X slower than Snowflake Standard Analytical queries

Features Not Yet Supported:

- Cloning (For HYBRID tables, cloning will require a copy or restore from backup instead of a Metadata operation that is used for Standard Tables)
- Clustering Keys
- Collations
- Data Retention Period
- Data sharing
- Dynamic Tables
- Fail-safe
- Materialized Views
- Periodic rekeying
- Query Acceleration Service
- Replication
- Search Optimization Service
- Snowpipe
- Streams
- Tri-secret secure encryption
- Time Travel
- UNDROP

Query acceleration service

Its similar to increasing the warehouse size dynamically which is useful for slow running queries.

https://thinketl.com/query-acceleration-service-in-snowflake/

1. Introduction

In our previous article on **Virtual Warehouses**, we have discussed about **Multi-cluster warehouses** which enables you to automatically scale out compute resources to manage concurrent queries during peak hours.

Similar to multi-cluster warehouses which lets you automatically scale out your compute resources horizontally (i.e. add more resources of same size), there is **no** feature in Snowflake which lets you automatically scale up your compute resources vertically (i.e. resizing of warehouse) to improve performance of slow-running queries.

Snowflake's solution to dynamically improve performance of slow-running queries is Query Acceleration Service (QAS). Let us understand more about this service in this article.

2. What is Query Acceleration Service in Snowflake?

> **The Query Acceleration Service is a feature that can be enabled for a Virtual Warehouse which accelerates parts of query workload automatically by offloading portions of query processing to the additional compute resources provided by the service. These additional resources provisioned by QAS work in parallel reducing the overall time spent in scanning and filtering.**

The queries that benefit from Query Acceleration Service are:

- Queries with large scans and selective filters.
- Workloads with unpredictable data volume per query.
- Adhoc analytics.

Note that Query Acceleration Service is an **Enterprise Edition (or higher)** feature.

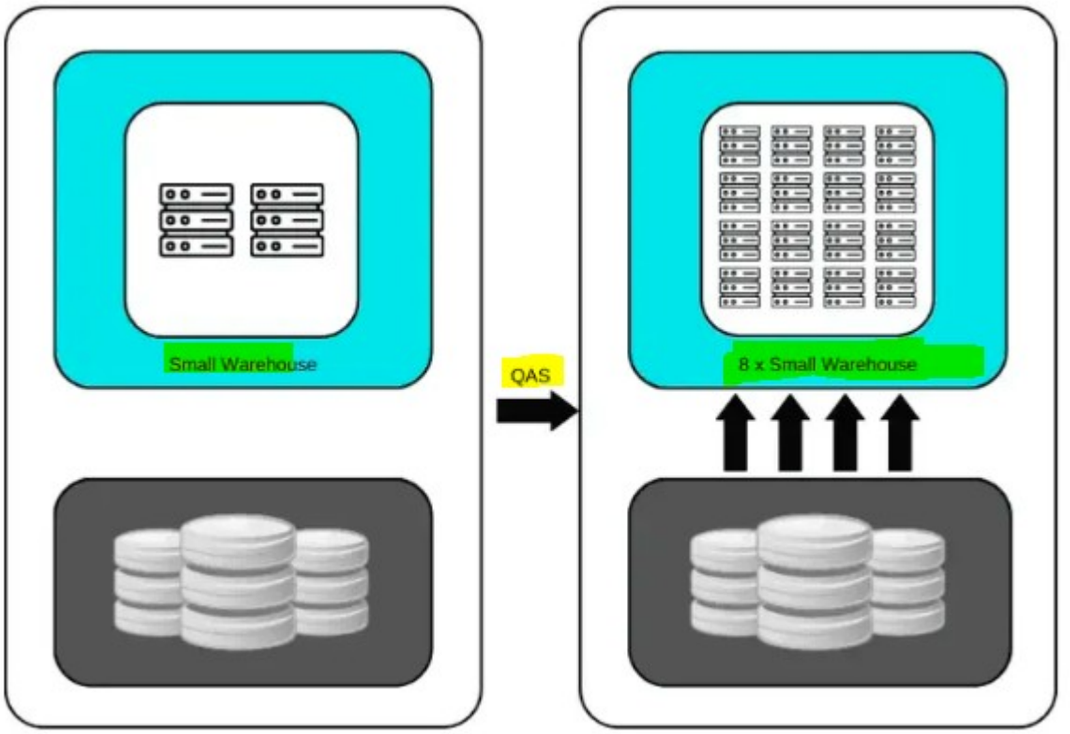
3. How Query Acceleration Service in Snowflake works?

Consider a huge query is running on an SMALL warehouse (2 credits per hour), it results in a poor query performance and also results in blocking all other queries from running.

While huge queries running on an X-LARGE warehouse will undoubtedly run faster, this is not an ideal solution when you have only a small number of large queries to run and many small queries. This leads to higher overall running costs as the smaller queries don't make full use of the compute resources, but the cluster is still charged at a rate of 16 credits per hour.

The Query Acceleration Service enabled on **an SMALL warehouse** can automatically detect when a huge query needs more resources and leases additional compute resources to complete the query, and then release the resources when the query is finished. There by reduces the overall cost compared to using a warehouse of bigger size.

The number of resources that the QAS can lease can be controlled using **Scale Factor** defined for QAS.



Extracting data using Small Warehouse with QAS enabled

The above illustration shows that an SMALL warehouse with QAS enabled up to 8X scale factor detects a huge query. Then the QAS deploys additional resources which helps in extracting data faster.

The Query Acceleration Service effectively functions as a group of resources that are temporarily deployable alongside your current warehouse and when needed, takes on some of the heavy lifting.

3. How to enable Query Acceleration Service in Snowflake?

4.

The following is the SQL statement that enables Query Acceleration Service while creating the Virtual Warehouse.

```
CREATE WAREHOUSE <warehouse_name> WITH
  WAREHOUSE_SIZE='<warehouse_size>'
  ENABLE_QUERY_ACCELERATION = true
  QUERY_ACCELERATION_MAX_SCALE_FACTOR = <upper_limit_scale_factor>
  INITIALLY_SUSPENDED = true
  AUTO_SUSPEND = 60;
```

The Query Acceleration Service can also be enabled for an existing Virtual Warehouse using the below ALTER statement.

```
ALTER WAREHOUSE <warehouse_name>
   ENABLE_QUERY_ACCELERATION = true
   QUERY_ACCELERATION_MAX_SCALE_FACTOR = <upper_limit_scale_factor>;
```

5. What is Scale Factor in Query Acceleration Service?

> The Scale Factor in Query Acceleration Service is a control mechanism which lets you select the maximum number of the compute resources a warehouse can lease for Query Acceleration. The scale factor is a **multiplier value** for number of compute resources of same warehouse size and cost.

For example, if you set a scale factor to 8 for a SMALL warehouse.

- The warehouse can lease compute resources up to 8 times the size of SMALL warehouse.
- As the SMALL warehouse costs 2 credits per hour, leasing these resources would cost up to 16 credits per hour (2 credits per hour X 8 times compute resources)

Few key points related to Scale Factor

- The default value of Scale Factor is 8 if it is not explicitly set.
- Setting the scale factor to 0 eliminates the upper bound limit and allows queries to lease as many resources as necessary and available to execute the query.
- The Query Acceleration Service is billed by the second, only when the service is in use. These credits are billed separately from warehouse usage.
- Snowflake automatically determines whether the query would benefit from using the Query Acceleration Service, and will only deploy this if it's estimated to improve query performance and overall throughput.

Which queries are supported by Query Acceleration Service?

Currently the Query Acceleration Service can only accelerate certain parts of query which includes fetching data and filtering out the results. However, these are the parts of the query which take most of the execution time. Also the amount of data that is being extracted must be huge for the query to be eligible for query acceleration.

The queries which include the following SQL commands are supported by QAS.

- SELECT
- INSERT (when the statement contains a SELECT statement)

- CREATE TABLE AS SELECT (CTAS)

The parts of query which does aggregation, sorting and other steps are not supported by Query Acceleration Service. Although this may be supported in upcoming releases.

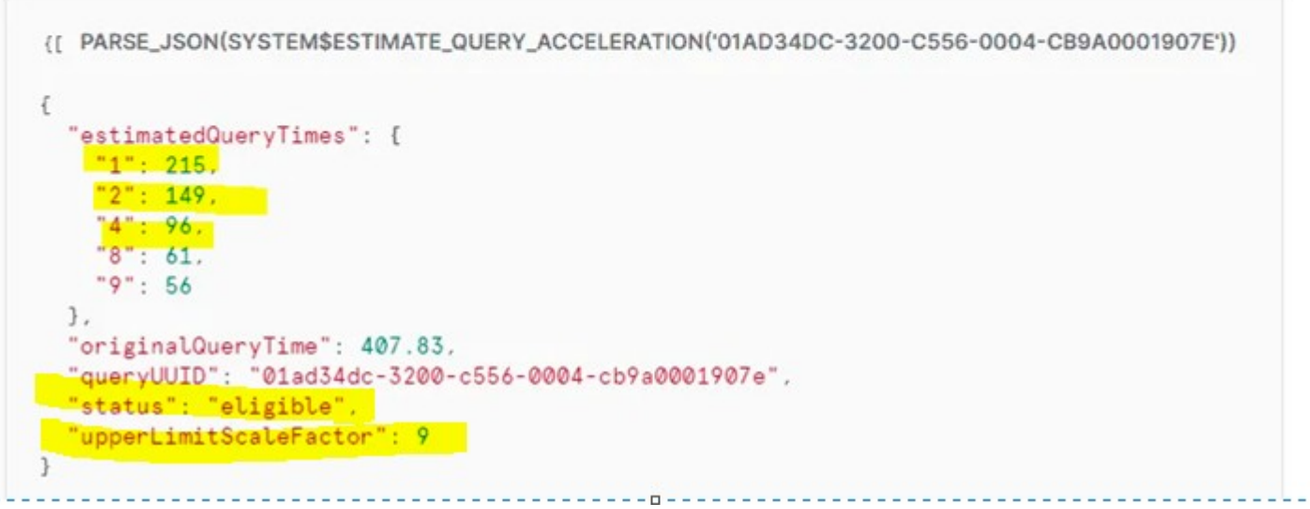
How to identify Queries that take advantage of Query Acceleration Service?

To identify if a query is eligible for Query Acceleration Service, you can use the **SYSTEM$ESTIMATE_QUERY_ACCELERATION** function as shown below.

```
SELECT PARSE_JSON(SYSTEM$ESTIMATE_QUERY_ACCELERATION('<query_id>'));
```

Below is the output using the query_id of the query which we ran using the DEMO_WH in step 6.1. which shows that the query is eligible for enabling Query Acceleration service. It also suggests that setting the scale factor to 9 for better performance. It also provides estimated query times for different scale factors.

```
SELECT PARSE_JSON(SYSTEM$ESTIMATE_QUERY_ACCELERATION('01ad34dc-3200-c556-0004-cb9a0001907e'));
```



The **QUERY_ACCELERATION_ELIGIBLE** view also identifies queries along with warehouses that might benefit from the query acceleration service.

```
SELECT * FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_ACCELERATION_ELIGIBLE;
```

```
SELECT * FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_ACCELERATION_ELIGIBLE
WHERE start_time >= DATEADD(hour, -1, CURRENT_TIMESTAMP());
```

Results | Chart

	QUERY_ID	QUERY_TEXT ···	WAREHOUSE_NAME	WAREHOUSE_SIZE
1	01ad355d-3200-c55f-0004-cb9a0001a0fe	select store.s_store_	DEMO_WH	Small

QUERY_ACCELERATION_ELIGIBLE view output

9. How to find the Billing cost of Query Acceleration?

Query Acceleration in Snowflake consumes credits for the amount of time the additional resources are leased.

The billing data of Query Acceleration Service can be found using the **QUERY_ACCELEARTION_HISTORY** view in **ACCOUNT_USAGE** schema as shown in below query.

```
SELECT warehouse_name,
       SUM(credits_used) AS total_credits_used
  FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_ACCELERATION_HISTORY
  WHERE start_time >= DATEADD(hour, -24, CURRENT_TIMESTAMP())
  GROUP BY 1
  ORDER BY 2 DESC;
```

The billing data of Query Acceleration Service can also be found using the **QUERY_ACCELEARTION_HISTORY** function in **INFORMATION_SCHEMA** as shown in below query.

```
SELECT start_time, end_time, credits_used, warehouse_name
  FROM TABLE(INFORMATION_SCHEMA.QUERY_ACCELERATION_HISTORY(
    date_range_start=>DATEADD(H, -24, CURRENT_TIMESTAMP)));
```

10. Conclusion

QAS is a great feature to improve the query performance of long running queries on the existing warehouse without the need of scaling up the warehouse size.

QAS is enabled only if Snowflake identifies that the query would be benefitted from additional compute resources. The QAS feature also comes with additional cost. Adjust the scale factor to control the amount of resources that a warehouse can lease.

Search optimization service

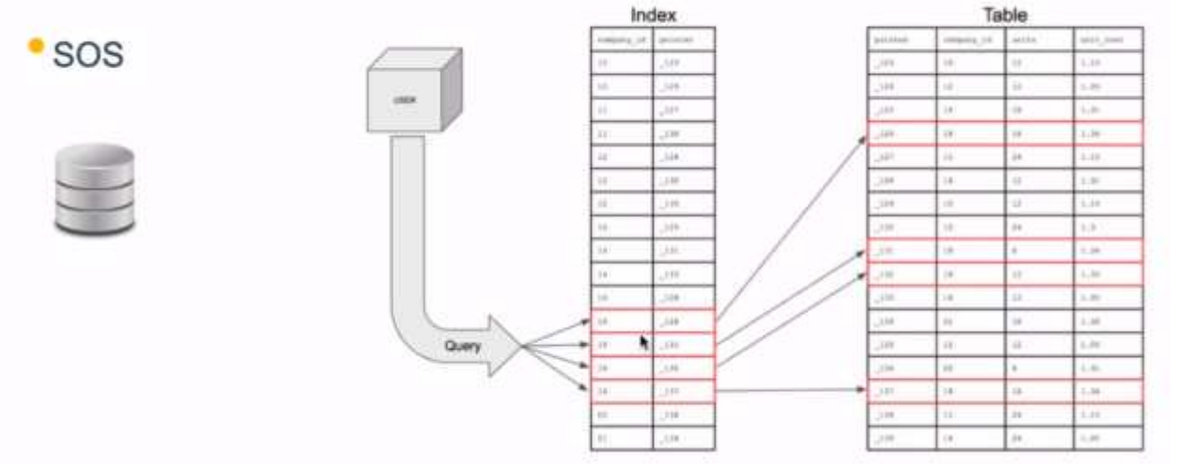
https://thinketl.com/search-optimization-service-in-snowflake/

1. What is Search Optimization Service in Snowflake?

The Search Optimization Service in Snowflake is a query optimization service that can enhance the performance of certain types of lookup and analytical queries which retrieves a small subset of results from large data volumes.

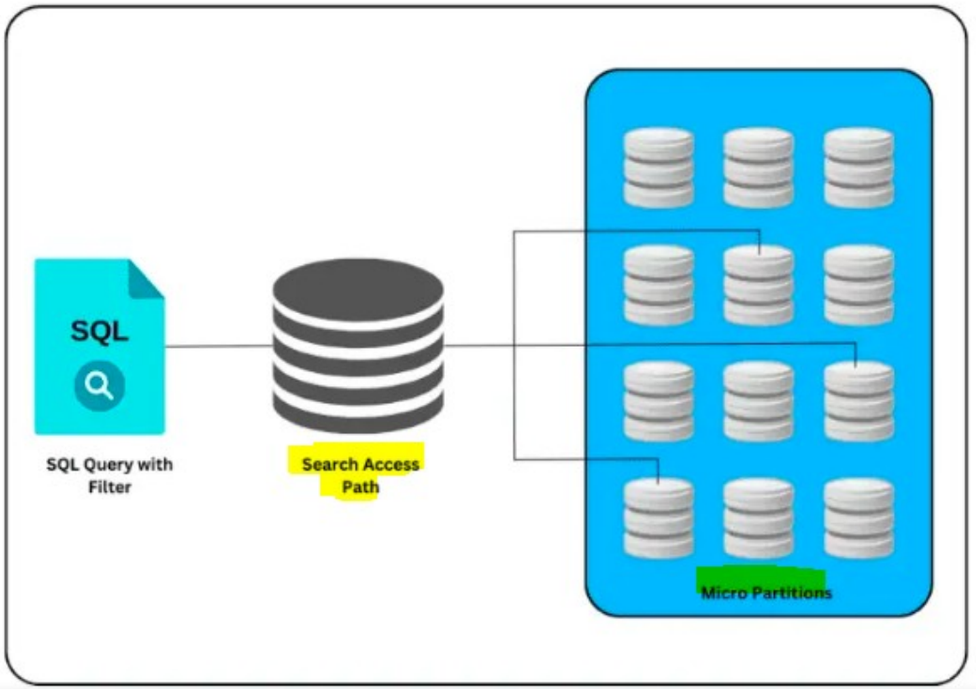
In this article let us discuss how search optimization service improves query performance, identify tables and columns on which search optimization can be enabled and implement it.

How SOS works.



2. How Search Optimization Service works in Snowflake?

When Search Optimization Service is enabled on a table, it creates an additional data set called **Search Access Path** that keeps tracks of all the micro partitions where the values of the table are stored. The Search Access Path improves query performance by reducing the amount of partitions scanned during the table scan operation instead of searching all the partitions of the table.



The following are the Key points related to search access paths

- When search optimization is enabled on a table, the process of populating data into search access path for the first time might take significant time depending on the size of the data.
- When data in the table is modified, the search access path is automatically updated by Snowflake maintenance service.
- There is additional cost involved for the storage and compute resources for maintaining the search access path.

3. Which queries take benefit of Search Optimization Service?

Search Optimization Service improves the performance of selective point lookup queries on tables that use

- Equality or IN Predicates
  - <column_name> = <constant>
  - <column_name> IN (<constant>,<constant>,…)
- Substrings and Regular Expressions
  - LIKE
  - RLIKE
  - REGEXP
  - REGEXP_LIKE …
- Fields in VARIANT Columns
- Geospatial Functions with GEOGRAPHY objects
- Conjunctions of above Supported Predicates (AND)
  - where <condition_x> and <condition_y>
- Disjunctions of above Supported Predicates (OR)
  - where <condition_x> or <condition_y>

4. How to identify tables and columns that benefit from Search Optimization Service?

The following are the recommended checks to consider a table and its columns for Search Optimization Service.

- The data volume of the table on which the query is executed is typically at least in few hundreds of GBs.
- The columns used in the query filter operation has at least 100K-200K distinct values.
- The query returns only a few rows with highly selective filters.
- The query typically runs for at least few seconds or longer.
- The table is either not clustered or the table is frequently queried on columns other than the cluster key columns.

5. How to enable Search Optimization Service for a table in Snowflake?

The Search Optimization Service can be enabled on a table using **ALTER TABLE ... ADD SEARCH OPTIMIZATION** command with or without the ON clause.

The columns on which search optimization to be enabled is specified in the ON clause along with a search method (ex: **EQUALITY** for equality and IN searches, **SUBSTRING** for substring searches and **GEO** for GEOGRAPHY searches.)

The following SQL statement enables search optimization on table t1 with equality search filters on columns c1, c2.

```
ALTER TABLE t1 ADD SEARCH OPTIMIZATION ON EQUALITY(c1, c2);
```

The following SQL statement enables search optimization on table t1 with substring searches on columns c3.

```
ALTER TABLE t1 ADD SEARCH OPTIMIZATION ON SUBSTRING(c3);
```

Multiple search methods can be configured for a table in the ON clause separated by commas.

The following SQL statement enables search optimization on table t1 with equality search filters on columns c1, c2 and substring searches on columns c3.

```
ALTER TABLE t1 ADD SEARCH OPTIMIZATION ON EQUALITY(c1, c2), SUBSTRING(c3);
```

Enabling the search optimization on a table without ON clause sets up search access paths using EQUALITY search method for all supported columns.

The following SQL statement enables search optimization on table t1 with equality filters on all supported columns.

```
ALTER TABLE t1 ADD SEARCH OPTIMIZATION;
```

6. How to identify if Search Optimization is configured on a table?

The following command can be used to identify if search optimization is configured on a table.

```
SHOW TABLES;
```

For the tables where search optimization is configured,

- The value of the field **search_optimization** will be **ON**.
- The **search_optimization_progress** field indicates the status of creation of search access path.
- The **search_optimization_bytes** field indicates the amount of storage consumed by the search access path built for the table.

7.2. Enabling Search Optimization on a table

Configuring search optimization on table LINEITEM_SOS for equality search filters on column l_order_key.

```
ALTER TABLE DEMO_DB.PUBLIC.LINEITEM_SOS ADD SEARCH OPTIMIZATION ON EQUALITY(l_orderkey);
```

7.3. Verify the Search Optimization Progress

When search optimization is configured on a table, the maintenance service starts creating the search access path for the table based on the column configured. This might take some time based on the volume of data.

Verify the search optimization progress using **SHOW TABLES** command as shown below.

```
SHOW TABLES LIKE 'LINEITEM%';
```

	name	search_optimization	search_optimization_progress	search_optimization_bytes
1	LINEITEM_NON_SOS	OFF	null	null
2	LINEITEM_SOS	ON	100	61,348,457,472

Verifying the Search Optimization

8. How to identify which columns are configured for Search Optimization in a table?

To identify the columns and their search optimization configuration in a table, use the **DESCRIBE** **SEARCH OPTIMIZATION** command.

The following SQL statement provides the search optimization configuration of table LINEITEM_SOS.

```
DESCRIBE SEARCH OPTIMIZATION ON DEMO_DB.PUBLIC.LINEITEM_SOS;
```

	expression_id	method	target	target_data_type	active
1	1	EQUALITY	L_ORDERKEY	NUMBER(38,0)	true
2	2	SUBSTRING	L_COMMENT	VARCHAR(44)	true
3	3	EQUALITY	L_COMMENT	VARCHAR(44)	true

Identifying columns configured for Search Optimization

The command returns a table with list of fields configured for search optimization, their search method and data type. The **active** fields indicates if the initial build of the search access path is finished.

9. How to Disable Search Optimization Service in Snowflake?

The Search Optimization Service can be disabled on a table using **ALTER TABLE … DROP SEARCH OPTIMIZATION** command.

The following SQL statement removes search optimization on table t1 with substring searches on columns c1.

```
ALTER TABLE t1 DROP SEARCH OPTIMIZATION ON SUBSTRING(c1);
```

The following SQL statement removes all search optimization methods configured on columns c1.

```
ALTER TABLE t1 DROP SEARCH OPTIMIZATION ON c1;
```

The following SQL statement removes all search optimization methods configured on all columns of table t1.

```
ALTER TABLE t1 DROP SEARCH OPTIMIZATION;
```

10. How to estimate billing costs of Search Optimization Service in Snowflake?

The Search Optimization Service in Snowflake incurs costs for both storage and compute resources.

- The Search Access Paths created by search optimization service on a table consumes storage and the storage cost depends on multiple factors like number of columns on which search optimization is configured and the number of distinct value of each column.
- Compute resources are utilized for the initial build of the search access paths when search optimization is enabled. Maintaining the search optimization service when data is modified in the table also requires resources.

To estimate the cost of adding search optimization to a table and configuring specific columns for search optimization, use the **SYSTEM$ESTIMATE_SEARCH_OPTIMIZATION_COSTS** function.

The following statement shows the estimated costs of adding search optimization to a table.

```
SELECT SYSTEM$ESTIMATE_SEARCH_OPTIMIZATION_COSTS('DEMO_DB.PUBLIC.LINEITEM_NON_SOS') AS
SOS_ESTIMATE;
```

The following statement shows the estimated costs of adding search optimization to a specific column of a table using EQUALITY search method.

```
SELECT SYSTEM$ESTIMATE_SEARCH_OPTIMIZATION_COSTS('DEMO_DB.PUBLIC.LINEITEM_NON_SOS',
'EQUALITY(L_ORDERKEY)') AS SOS_ESTIMATE;
```

SOS_ESTIMATE


```
{
  "tableName" : "LINEITEM_NON_SOS",
  "searchOptimizationEnabled" : false,
  "costPositions" : [ {
    "name" : "BuildCosts",
    "costs" : {
      "value" : 1.981617,
      "unit" : "Credits"
    },
    "computationMethod" : "Estimated",
    "comment" : "estimated via sampling"
  }, {
    "name" : "StorageCosts",
    "costs" : {
      "value" : 0.036371,
      "unit" : "TB",
      "perTimeUnit" : "MONTH"
    },
    "computationMethod" : "Estimated",
    "comment" : "estimated via sampling"
  }, {
    "name" : "MaintenanceCosts",
    "computationMethod" : "NotAvailable",
    "comment" : "Insufficient data to compute estimate for
maintenance cost. Table is too young. Requires 7 day(s) of
history."
  } ]
```


Cost estimate for enabling Search Optimization with Equality search method on a column

Snowflake tags

https://thinketl.com/tagging-in-snowflake/

4. How to Create Tags in Snowflake?

Tags in Snowflake can be created using **CREATE TAG** statement as shown below.

```
CREATE OR REPLACE TAG SENSITIVE_DATA;
CREATE OR REPLACE TAG STAFF_DATA;
```

> ***Tags can also be created with a list of allowed values.***

The below example shows creating a tag named PII_DATA with **Names**, **Contact Details** and **Address** as allowed values.

```
CREATE OR REPLACE TAG PII_DATA
      ALLOWED_VALUES 'Names', 'Contact Details','Address';
```

5. How to find the allowed values for a Tag?

To find the list of allowed string values for a given tag, below functions can be used.

- **GET_DDL**
- **SYSTEM$GET TAG ALLOWED VALUES**

6.3. Assigning Tags at Table level

When a tag is assigned at a table level, all the columns the table will inherit the tag.

The below example assigns the tag **PII_DATA** with tag value as **Address** on the table named `LOCATIONS`.

```
ALTER TABLE demo_db.public.locations SET TAG PII_DATA = 'Address';
```

6.4. Assigning Tags at Column level

To set a tag on an existing column, use the **ALTER TABLE … MODIFY COLUMN** command for a table column

The below example assigns the tag **PII_DATA** on multiple columns with different tag values on the table named `EMPLOYEES`.

```
ALTER TABLE employees MODIFY COLUMN first_name SET TAG PII_DATA = 'Names';
ALTER TABLE employees MODIFY COLUMN last_name SET TAG PII_DATA = 'Names';

ALTER TABLE employees MODIFY COLUMN email SET TAG PII_DATA = 'Contact Details';
ALTER TABLE employees MODIFY COLUMN phone_number SET TAG PII_DATA = 'Contact Details';
```

> ***It is recommended to apply tags at the more granular level possible for the requirement. Applying same tag on all objects defeats the whole purpose of tags.***

7.2. TAGS View

The **TAGS** view in the Account Usage schema of Snowflake database provides information of all the tags in your Snowflake account including the deleted tags.

```
SELECT * FROM SNOWFLAKE.ACCOUNT_USAGE.TAGS
ORDER BY TAG_NAME;
```

	TAG_NAME	TAG_SCHEMA	TAG_DATABASE	TAG_OWNER ···	ALLOWED_VALUES	OWNER_ROLE_TYPE
1	PII_DATA	PUBLIC	DEMO_DB	DATA_ENGINEER	["Address", "Contact Details", "Names"]	ROLE
2	SENSITIVE_DATA	PUBLIC	DEMO_DB	DATA_ENGINEER	null	ROLE
3	STAFF_DATA	PUBLIC	DEMO_DB	DATA_ENGINEER	null	ROLE

7.3. TAG_REFERENCES View

The **TAG_REFERENCES** view in the Account Usage schema of Snowflake database provides information of all the objects in your Snowflake account that are assigned with a tag and a tag value.

```
SELECT * FROM SNOWFLAKE.ACCOUNT_USAGE.TAG_REFERENCES
ORDER BY DOMAIN, COLUMN_ID;
```

	TAG_NAME	TAG_VALUE	OBJECT_NAME	DOMAIN	COLUMN_ID	COLUMN_NAME
1	PII_DATA	Names	EMPLOYEES	COLUMN	2	FIRST_NAME
2	PII_DATA	Names	EMPLOYEES	COLUMN	3	LAST_NAME
3	PII_DATA	Contact Details	EMPLOYEES	COLUMN	4	EMAIL
4	PII_DATA	Contact Details	EMPLOYEES	COLUMN	5	PHONE_NUMBER
5	SENSITIVE_DATA	confidential	DEMO_DB	DATABASE	null	null
6	STAFF_DATA	HR	PUBLIC	SCHEMA	null	null
7	PII_DATA	Address	LOCATIONS	TABLE	null	null

The above output shows all the tags assigned at **DATABASE**, **SCHEMA**, **TABLE** and **COLUMN** levels at the account level.

ROW ACCESS POLICY

https://thinketl.com/row-level-security-using-row-access-policies-in-snowflake/

2. What are Row Access Policies in Snowflake?

A Row Access Policy is a schema-level object that determines whether a given row in a table or view can be viewed by a user using the following types of statements.

1. **SELECT statements**
2. **Rows selected by UPDATE, DELETE, and MERGE statements.**

The row access policy should be added to a table or a view binding with a column present inside them. A row access policy can be added to a table or view either when the object is created or after the object is created.

3. Steps to implement Row-Level Security using Row Access Policies in Snowflake

Follow below steps to implement Row-Level Security using Row Access Policies in Snowflake.

1. Create a table to apply Row-Level Security
2. Create a Role Mapping table
3. Create a Row Access Policy
4. Add the Row Access Policy to a table
5. Create Custom Roles and their Role Hierarchy
6. Grant SELECT privilege on table to custom roles
7. Grant USAGE privilege on virtual warehouse to custom roles
8. Assign Custom Roles to Users
9. Query and verify Row-Level Security on table using custom roles
10. Revoke privileges on role mapping table to custom roles

The below SQL statements creates a table named *employees* with required sample data in *hr* schema of *analytics* database.

```
use role SYSADMIN;

create or replace database analytics;

create or replace schema analytics.hr;

create or replace table analytics.hr.employees(
    employee_id number,
    first_name varchar(50),
    last_name varchar(50),
    email varchar(50),
    hire_date date,
    country varchar(50)
);

INSERT INTO analytics.hr.employees(employee_id,first_name,last_name,email,hire_date,country)
(100,'Steven','King','SKING@outlook.com','2013-06-17','US'),
(101,'Neena','Kochhar','NKOCHHAR@outlook.com','2015-09-21','US'),
(102,'Lex','De Haan','LDEHAAN@outlook.com','2011-01-13','US'),
(103,'Alexander','Hunold','AHUNOLD@outlook.com','2016-01-03','UK'),
```

3.2. Create a Role Mapping table

The below SQL statements creates mapping table named ***role_mapping*** which stores the country and corresponding role to be assigned for the users of that country as shown below.

```
use role SYSADMIN;

create or replace table analytics.hr.role_mapping(
    country varchar(50),
    role_name varchar(50)
);

INSERT INTO analytics.hr.role_mapping(country, role_name) VALUES
('US','DATA_ANALYST_ROLE_US'),
('UK','DATA_ANALYST_ROLE_UK'),
('CA','DATA_ANALYST_ROLE_CA')
;
```

3.3. Create a Row Access Policy

The below SQL statement creates a Row Access Policy with following two conditions.

1. User with SYSADMIN role can query all rows of the table.
2. User with DATA_ANALYST roles can query only rows belonging to their country based on the role mapping table.

```
use role SYSADMIN;
create or replace row access policy analytics.hr.country_role_policy as (country_name
varchar) returns boolean ->
  'SYSADMIN' = current_role()
      or exists (
            select 1 from role_mapping
              where role_name = current_role()
                and country = country_name
          )
;
```

In the above statement:

country_role_policy specifies the name of the policy.

country_name is the signature of the row access policy which specifies the field and data type of the mapping table to which it links.

returns boolean -> specifies the application of the row access policy.

'SYSADMIN' = current_role() is the first condition of row access policy which allows users with SYSDAMIN role to view all rows of the table.

or exists … is the second condition of the row access policy expression which uses a subquery. The subquery requires the CURRENT_ROLE to be the custom role which specifies the country through role mapping table. This is used by row access policy to limit the rows to be returned for the query executed by user.

3.4. Add the Row Access Policy to a table

The below SQL statement adds the row access policy named *country_role_policy* to the table *employees* on *country* field.

```
use role SYSADMIN;

alter table analytics.hr.employees
add row access policy analytics.hr.country_role_policy on (country);
```

3.9. Query and verify Row-Level Security on table using custom roles

Let us verify the data returned for each user when queried on the same table.

The below image shows that for user with role *DATA_ANALYST_ROLE_US* when queried on the table *employees*, the data returned is only from country US.

```
115  use role DATA_ANALYST_ROLE_US;
116  select * from analytics.hr.employees;
```

Objects | Editor | Results | Chart

	EMPLOYEE_ID	FIRST_NAME···	LAST_NAME	EMAIL	HIRE_DATE	COUNTRY
1	100	Steven	King	SKING@outlook.com	2013-06-17	US
2	101	Neena	Kochhar	NKOCHHAR@outlook.com	2015-09-21	US
3	102	Lex	De Haan	LDEHAAN@outlook.com	2011-01-13	US

Query returning only US data when queried with DATA_ANALYST_ROLE_US role

The below image shows that when the user with role *SYSADMIN* queries on the table *employees*, all rows are returned.

```
125  use role SYSADMIN;
126  select * from analytics.hr.employees;
```

Objects | Editor | Results | Chart

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME ···	EMAIL	HIRE_DATE	COUNTRY
1	100	Steven	King	SKING@outlook.com	2013-06-17	US
2	101	Neena	Kochhar	NKOCHHAR@outlook.com	2015-09-21	US
3	102	Lex	De Haan	LDEHAAN@outlook.com	2011-01-13	US
4	103	Alexander	Hunold	AHUNOLD@outlook.com	2016-01-03	UK
5	104	Bruce	Ernst	BERNST@outlook.com	2017-05-21	UK
6	105	David	Austin	DAUSTIN@outlook.com	2015-06-25	UK
7	106	Valli	Pataballa	VPATABAL@outlook.com	2016-02-05	CA
8	107	Diana	Lorentz	DLORENTZ@outlook.com	2017-02-07	CA
9	108	Nancy	Greenberg	NGREENBE@outlook.com	2012-08-17	CA

The below SQL statement revokes all privileges on table *role_mapping* to the custom roles.

```
use role SYSADMIN;

revoke all privileges on table analytics.hr.role_mapping from role DATA_ANALYST_ROLE_US;
revoke all privileges on table analytics.hr.role_mapping from role DATA_ANALYST_ROLE_UK;
revoke all privileges on table analytics.hr.role_mapping from role DATA_ANALYST_ROLE_CA;
```

4. How to Remove a Row Access Policy on a table in Snowflake?

The below SQL statement removes a row access policy on a table.

```
alter table <table_name> drop row access policy <policy_name>;
```

The below SQL statement removes all row access policy associations from a table.

```
alter table <table_name> drop all row access policies;
```

```
show row access policies;
```

```
show row access policies;
```

```
5  SHOW ROW ACCESS POLICIES;
```

Objects | Editor | Results | Chart

	name	database_name	schema_name	kind	owner
1	COUNTRY_ROLE_POLICY	ANALYTICS	HR	ROW_ACCESS_POLICY	SYSADMIN

Show Row Access Policies – Listing all Row Access Policies

5.2. DESCRIBE ROW ACCESS POLICY

Describes the current definition of a row access policy, including the creation date, name, data type, and SQL expression.

The below SQL statement extracts information of the row access policy *country_role_policy*.

```
describe row access policy country_role_policy;
```

```
5  DESCRIBE ROW ACCESS POLICY COUNTRY_ROLE_POLICY;
```

Objects | Editor | Results | Chart

	name	signature	return_type	body
1	COUNTRY_ROLE_POLICY	(COUNTRY_NAME VARCHAR)	BOOLEAN	'SYSADMIN' = current_role() or exists

Describe Row Access Policy – Extracting details of a Row Access Policy

8. How to Drop a Row Access Policy in Snowflake?

A Row Access Policy cannot be dropped successfully if it is currently attached to a resource. Before executing a DROP statement, detach the row access policy from the table or view.
Follow below steps to drop a row access policy in Snowflake

1. Find the objects on which the row access policy is attached.

The below SQL statement lists all the objects on which row access policy named *country_role_policy* is attached.

```
select * from table(information_schema.policy_references(policy_name=>'country_role_policy'));
```

```
1  select * from
   table(information_schema.policy_references(policy_name=>'country_role_policy'));
```

Objects | Editor | Results | Chart

	POLICY_NAME	POLICY_KIND	REF_ENTITY_NAME	REF_ARG_COLUMN_	POLICY_STATUS
1	COUNTRY_ROLE_POLICY	ROW_ACCESS_POLICY	EMPLOYEES	["COUNTRY"]	ACTIVE

Finding all objects on which Role Access Policy is applied

2. Remove the row access policy from all the tables and views to which it is associated. (*refer section-4 of the article*)

3. Drop the row access policy.

The below SQL statement drops row access policy named *country_role_policy*.

```
drop row access policy country_role_policy;
```

Query optimization

Storage considerations.

- Automatic clustering.
- Search optimization.
- Materialized views.

Dynamic tables

A Dynamic table materializes the result of a query that you specify. It can track the changes in the query data you specify and refresh the materialized results incrementally through an automated process.

To incrementally load data from a base table into a target table, define the target table as a dynamic table and specify the SQL statement that performs the transformation on the base table. The dynamic table eliminates the additional step of identifying and merging changes from the base table, as the entire process is automatically performed within the dynamic table.

Dynamic tables support **Time Travel**, **Masking**, **Tagging**, **Replication** etc. just like a standard Snowflake table

3. How to Create Snowflake Dynamic Tables?

Below is the syntax of creating Dynamic Tables in Snowflake.

```
CREATE OR REPLACE DYNAMIC TABLE <name>
  TARGET_LAG = { '<num> { seconds | minutes | hours | days }' | DOWNSTREAM }
  WAREHOUSE = <warehouse_name>
AS
  <query>
```

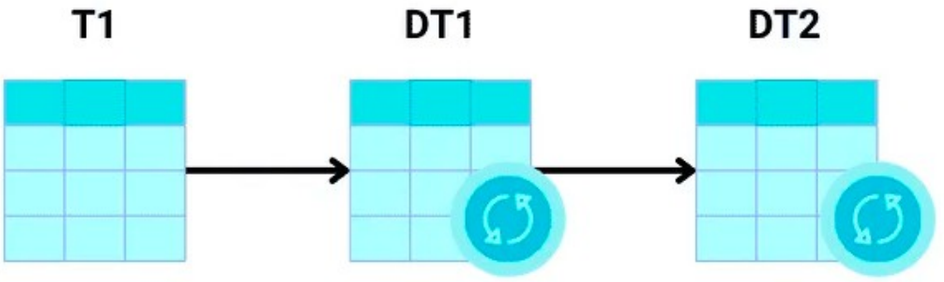
1. <name>: Name of the dynamic table.

2. TARGET_LAG: Specifies the lag between the dynamic table and the base table on which the dynamic table is built.

The value of TARGET_LAG can be specified in two different ways.

- **'<num> { seconds | minutes | hours | days }'** : Specifies the maximum amount of time that the dynamic table's content should lag behind updates to the base tables.
  Example: 1 minute, 7 hours, 2 days etc.
- **DOWNSTREAM:** Specifies that a different dynamic table is built based on the current dynamic table, and the current dynamic table refreshes on demand, when the downstream dynamic table need to refresh.

For example consider Dynamic Table 2 (**DT2**) is defined based on Dynamic Table 1 (**DT1**) which is based on table (**T1**).



Let's say if the target lag is set to 5 minutes for DT2, defining target lag as DOWNSTREAM for DT1 infers that DT1 gets its lag from DT2 and is updated every time DT2 refreshes.

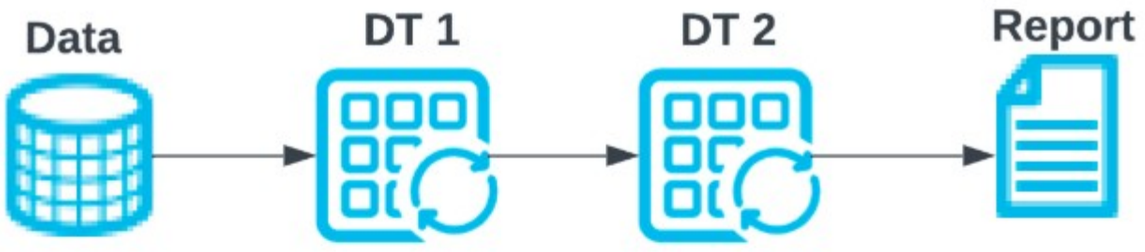
Dynamic table refresh modes

The dynamic table refresh process operates in one of two ways:

1. ***Incremental refresh:*** This automated process analyzes the dynamic table's query and calculates changes since the last refresh. It then merges these changes into the table. See Supported queries in incremental refresh for details on supported queries.
2. ***Full refresh:*** When the automated process can't perform an incremental refresh, it conducts a full refresh. This involves executing the query for the dynamic table and completely replacing the previous materialized results.

The constructs used in the query determine whether an incremental refresh can be used. After you create a dynamic table, you can monitor the table to determine whether incremental or full refreshes are used to update that table.

Consider the following example where Dynamic Table 2 (DT2) is defined based on Dynamic Table 1 (DT1). DT2 must read from DT1 to materialize its contents. In addition, a report consumes DT2 data via a query.



The following results are possible, depending on how each dynamic table specifies its lag:

Dynamic Table 1 (DT1)	Dynamic Table 2 (DT2)	Refresh results
`TARGET_LAG = DOWNSTREAM`	`TARGET_LAG = 10minutes`	DT2 is updated at most every 10 minutes. DT1 infers its lag from DT2 and is updated every time DT2 requires updates.
`TARGET_LAG = 10minutes`	`TARGET_LAG = DOWNSTREAM`	This scenario should be avoided. The report query will not receive any data. DT 1 is frequently refreshed and DT2 is not refreshed as there's no dynamic table that's based on DT2.
`TARGET_LAG = 5minutes`	`TARGET_LAG = 10minutes`	DT2 is updated approximately every 10 minutes with data from DT1 that's at most 5 minutes old.
`TARGET_LAG = DOWNSTREAM`	`TARGET_LAG = DOWNSTREAM`	DT2 is not refreshed periodically because DT1 has no downstream children with a defined lag.

Differences Between Snowflake Dynamic Tables and Materialized Views

Materialized Views	Dynamic Tables
A Materialized View cannot use a complex SQL query with joins or nested views.	A Dynamic table can be based on a complex query, including one with joins and unions.
A Materialized View can be built using only a single base table.	A Dynamic table can be built using multiple base tables including other dynamic tables.
A Materialized View always returns the current data when executed.	A Materialized View returns the data latest up to the target lag time.

Dynamic tables General System Limitations

- **Account Limits:** A single Snowflake account is capped at a maximum of **50,000** dynamic tables.
- **No Temporary Tables:** You cannot create a `TEMPORARY` or `VOLATILE` dynamic table; they must be permanent or transient.
- **No Truncate:** You cannot use the `TRUNCATE` command on a dynamic table.
- **Source Restrictions:** Dynamic tables **cannot** read directly from:
  - External tables
  - Directory tables
  - Streams
  - Materialized Views

2. SQL & Query Constraints

Certain SQL constructs are unsupported or behave differently:

- **Dynamic SQL:** You cannot use session variables or unbound variables in the table definition.

Dynamic SQL (Session Variables)

Dynamic Tables must be "self-contained." If a table definition relied on a session variable, Snowflake wouldn't know what value to use when the background refresh process runs (since there is no active user session).

- ❌ **Unsupported:**

-- This fails because $region_name might change or not exist during background refresh

CREATE OR REPLACE DYNAMIC TABLE sales_by_region

TARGET_LAG = '5 minutes'

WAREHOUSE = my_wh

AS

SELECT * FROM raw_sales WHERE region = $region_name;

✅ Workaround: Hardcode the value or join against a "Configuration Table" that contains the metadata you need.

- **User-Defined Functions (UDFs):** * UDFs written in Python, Java, or Javascript that are marked as `VOLATILE` are not supported.
    - SQL UDFs containing subqueries are blocked.
- **Subqueries:** Subqueries outside of the `FROM` clause (e.g., in a `WHERE EXISTS` or `WHERE IN` block) are generally not supported.

Snowflake currently requires the query structure to be very "flat" to map out the incremental data flow. Subqueries in the `WHERE` or `SELECT` clauses (Scalar Subqueries) are difficult to refresh incrementally.

- **❌ Unsupported (In WHERE/SELECT):**

SELECT

order_id,

(SELECT name FROM customers c WHERE c.id = o.cust_id) as cust_name -- Scalar subquery in SELECT

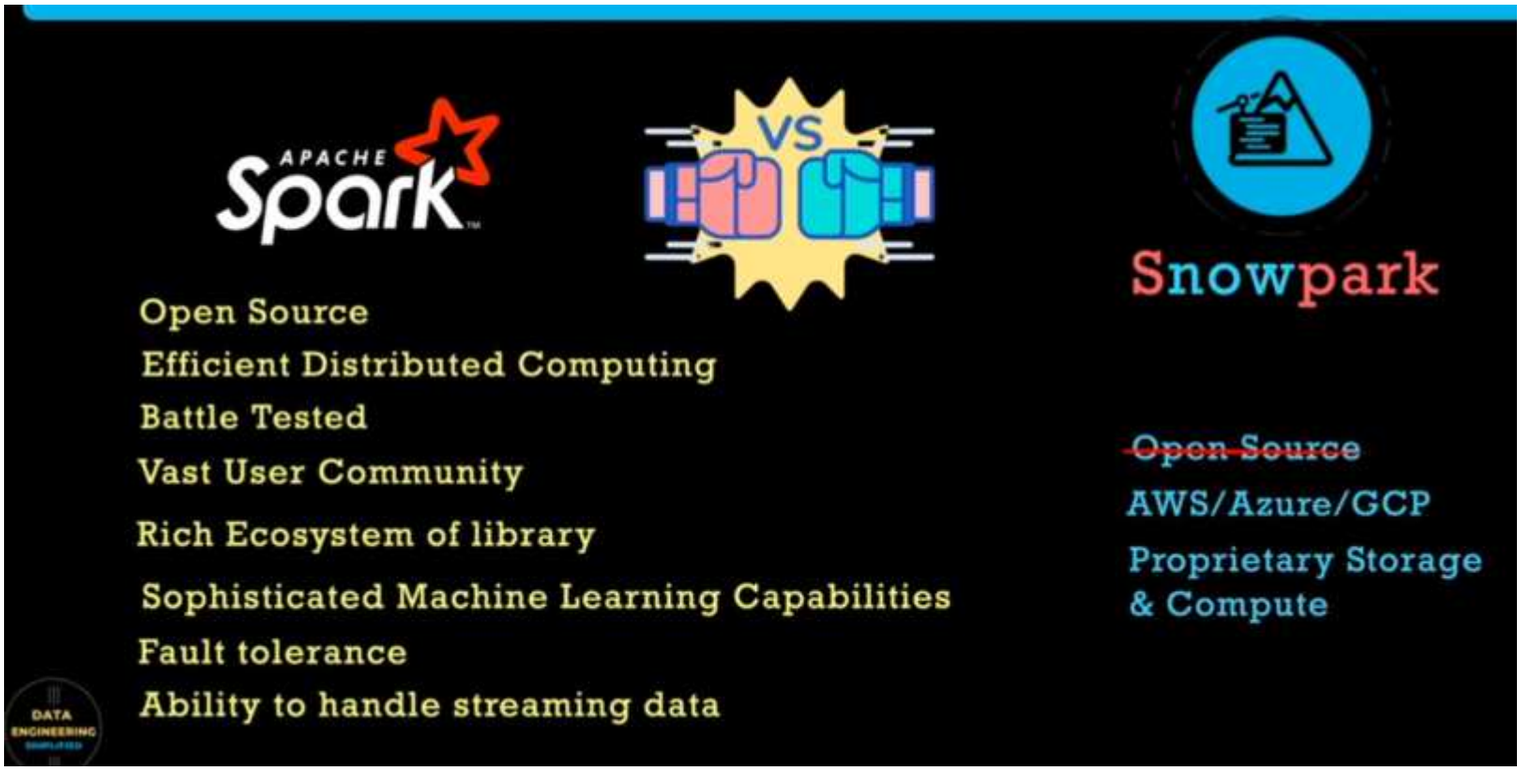
FROM orders o

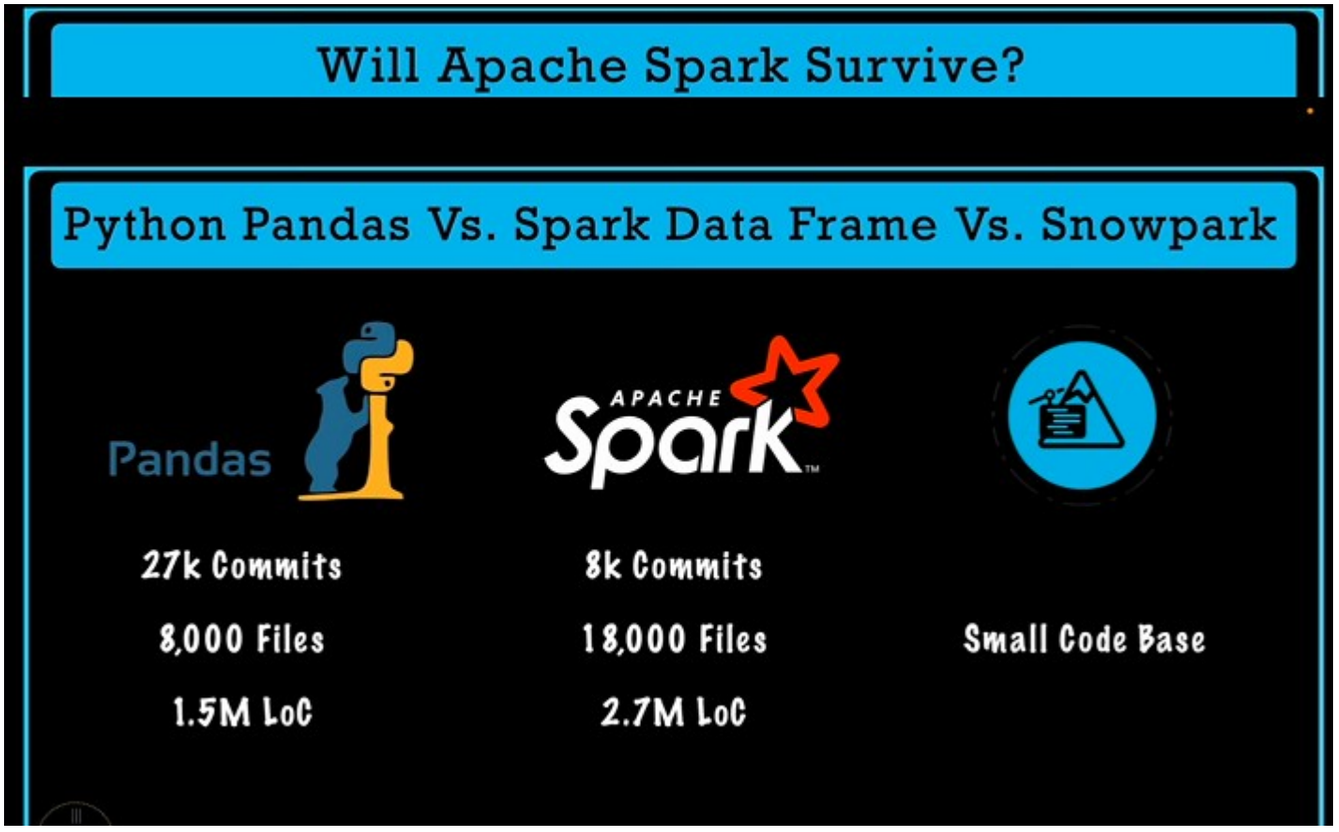
WHERE store_id IN (SELECT id FROM stores WHERE city = 'Atlanta'); -- Subquery in WHERE

✅ **Workaround (Use Joins): Almost any subquery can be rewritten as a JOIN, which *is* supported.**

- **Shared Data:** If you are ingesting data from a share, your query cannot select from a shared dynamic table or a shared secure view that references another upstream dynamic table.
  - o

Snowpark





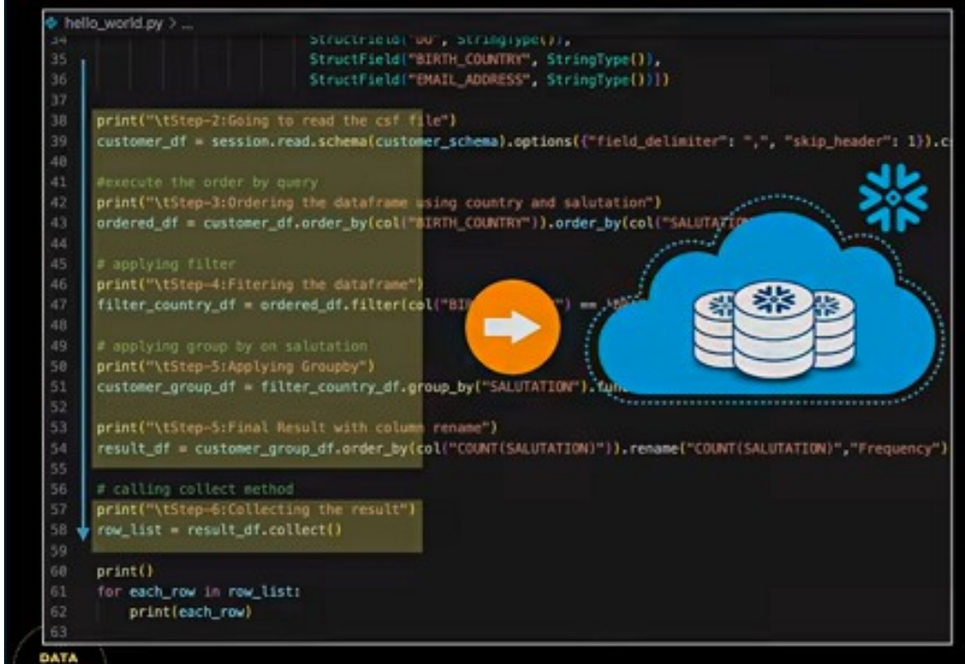
Developer > Snowpark API

Snowpark API

The Snowpark library provides an intuitive library for querying and processing data at scale in Snowflake. Using a library for any of three languages, you can build applications that process data in Snowflake without moving data to the system where your application code runs, and process at scale as part of the elastic and serverless Snowflake engine.

Snowflake currently provides Snowpark libraries for three languages: Java, Python, and Scala.

What Has Happened Under The Hood



Scan The Python File

While Running The Program

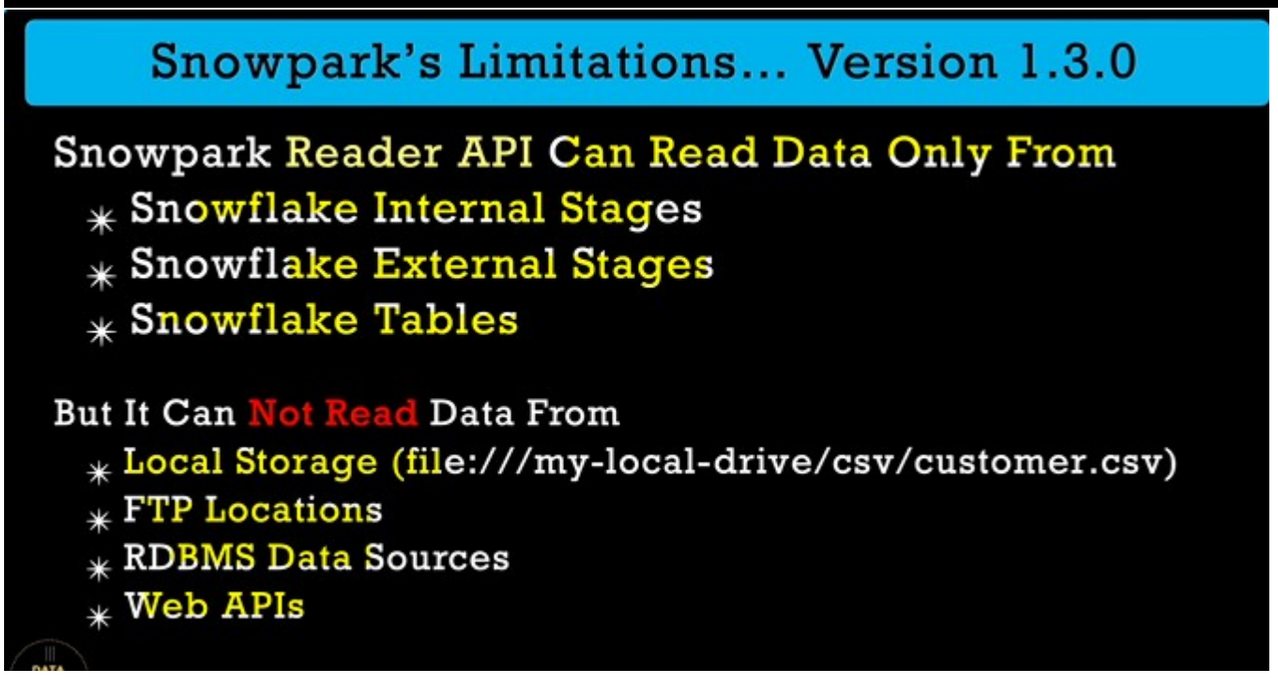
Identify Dataframe Actions

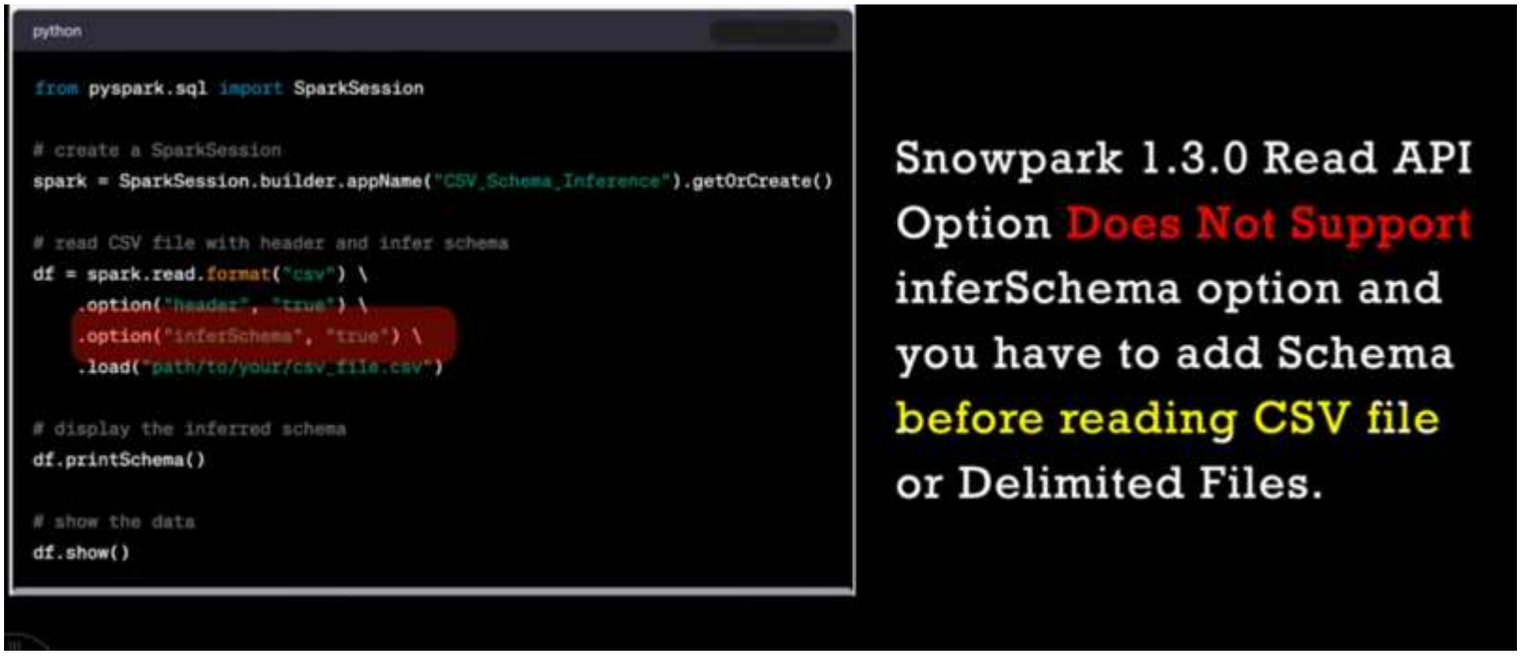
Build SQL From DF Transformation Steps

Using API Call, Executed The SQL Query In Snowflake VWH

What Is Snowpark In Snowflake?

A Translator Library, That Translates Your Programming Construct Into SQL Statement & Run Them In Snowflake Virtual Warehouse Using API Call





For csv files inferschema not supported

For parquet avro schems is attached so ingerschema will work

Snowpark's Limitations... Version 1.3.0

```
26  # reading customer data from internal stage
27  customer_schema = StructType([StructField("SALUTATION", StringType()),
28                                StructField("FIRST_NAME", StringType()),
29                                StructField("LAST_NAME", StringType()),
30                                StructField("DOB", StringType()),
31                                StructField("BIRTH_COUNTRY", StringType()),
32                                StructField("EMAIL_ADDRESS", StringType())])
33
34  print("\tStep-2:Going to read the csv file user stg location")
35  usr_stg_df = session.read \
36              .schema(customer_schema) \
37              .options({"field_delimiter": ",", "skip_header": 1}) \
38              .csv("@%customers/data_0_0_2.csv")
```

In below image which are highlighted in yellow colour are spark function that functions are not supported by snowpark version 1.3.0

Snowpark's Limitations... Version 1.3.0

df.alias(alias)
df.approxQuantile(..)
df.cache(..)
df.checkpoint(..)
df.coalesce(..)
df.colRegex(..)
df.createGlobalTemp(..)
df.createOrReplaceGlobalTempView(..)
df.createTempView(..)
df.crossJoin(..)
df.exceptAll(..)
df.foreach(f)
df.foreachPartition(..)
df.freqItems(..)
df.groupby(..)
df.head(..)
df.hint(...)
df.inputFiles()

CREATE SNOWPARK DATA FRAME USING PANDAS DATAFRAME

https://medium.com/snowflake/your-cheatsheet-to-snowflake-snowpark-dataframes-using-python-e5ec8709d5d7

```
session = Session.builder.configs(connection_parameters).create()
import pandas as pd
test = session.create_dataframe(pd.DataFrame([(1, 2, 3, 4,5)], columns=["a", "b", "c", "d","e"]))
test.show()
type(test)
test2 = session.table("DEMO_DB.PUBLIC.SNOWPARK_TEMP_TABLE_GDGL5S36VF")
```

- Pandas dataframe will create temporary table.
- Dataframe type will be Table.

Setting Up Session

The first step in using the library is establishing a session with the Snowflake database.

```
from snowflake.snowpark import Session
```

3. Call the `create` method of `builder` establishing the session.

```
# Constructing Dict for Connection Params
conn_config = {
    "account": "******.central-india.azure",
    "user": "******",
    "password": "************",
    "role" : "accountadmin",
    "warehouse" : "compute_wh",
    "database" : "snowflake_alert",
    "schema" : "edw"
}

#Invoking Snowpark Session for Establishing Connection
conn = Session.builder.configs(conn_config).create()
```

Using Snowpark Session SQL to query data from snowflake:

```
#.. Will not query data for below on snowflake
query_1 = conn.sql("select * from tb_catalog_sales limit 10")

#.. Will query data for below on snowflake because of **collect()**
query_2 = conn.sql("select * from tb_catalog_sales limit 20").collect()

#.. Will query data for query_1 because of **show()**
#query_1.show()
```

Creating Snowpark Dataframe by directly reading a table

```
df_tbl_read = conn.table("edw.tb_catalog_sales")
#df_tbl_read.show()
```

Creating Snowpark Dataframe by reading data from Stage

```
#.. Creating Dataframe by reading data from Stage

df_stg_read = conn.read.schema(StructType([StructField("name", StringType()),StructField("url",
StringType()),StructField("username", StringType()),StructField("password", StringType())])).csv(
"@test_stage/test_file.csv")
#df_stg_read.show()
```

Creating Snowpark Dataframe by Joining two tables/dataframes

```
#.. Creating Dataframe by Joining two tables/dataframes
df_tbl2_read = conn.table("edw.tb_catalog_sales_logs")
df_join = df_tbl_read.join(df_tbl2_read, df_tbl_read["CS_ITEM_SK"] == df_tbl2_read["CS_ITEM_SK"])
#df_join.show()
```

Performing operations on a DataFrame

Broadly, the operations on DataFrame can be divided into two types:

- **Transformations** produce a new DataFrame from one or more existing DataFrames. Note that transformations are lazy and don't cause the DataFrame to be evaluated. If the API does not provide a method to express the SQL that you want to use, you can use `functions.sqlExpr()` as a workaround.
- **Actions** cause the DataFrame to be evaluated. When you call a method that performs an action, Snowpark sends the SQL query for the DataFrame to the server for evaluation.

Transforming a DataFrame

Using Select Method to create a new dataframe with specific columns from existing DF

```
df_tbl_read = conn.table("edw.tb_catalog_sales")

#.. Using Select Method to create new Dataframe with specific columns from existing DF

df_select1 = df_tbl_read.select(col("CS_SOLD_TIME_SK"))
df_select2 = df_tbl_read.select(col("CS_SOLD_TIME_SK").substr(0, 3).as_("column1"))
df_select3 = df_tbl_read.select(col("CS_SOLD_TIME_SK").as_("column1"),col("CS_BILL_CDEMO_SK"))
#df_select1.show()
#df_select2.show()
#df_select3.show()
```

Using **Filter Method** to filter data (Similar to Where Clause)

```
#.. Using Filter Method to filter data (Similar to Where Clause)
#.. AND condition - &
#.. OR Condition - |

df_filter = df_tbl_read.filter((col("CS_SOLD_TIME_SK")==46616)&(col("CS_BILL_CDEMO_SK")==1642927)).select
(col("CS_EXT_WHOLESALE_COST"))
#df_filter.show()
```

Using the **SORT Method** to Order the data

```
#.. Using SORT Method to Order the data


df_sort1 = df_tbl_read.filter((col("CS_QUANTITY").isNotNull()) & (col("CS_WHOLESALE_COST").
isNotNull())).sort(col("CS_QUANTITY").asc(),col("CS_WHOLESALE_COST").desc()).select(col(
"CS_ORDER_NUMBER"),col("CS_QUANTITY"),col("CS_WHOLESALE_COST"),col("CS_LIST_PRICE"))
#df_sort1.show()
df_sort2 = df_tbl_read.filter((col("CS_QUANTITY").isNotNull()) & (col("CS_WHOLESALE_COST").
isNotNull())).sort([col("CS_QUANTITY"),col("CS_WHOLESALE_COST").desc()],ascending=[1,1]).select(
col("CS_ORDER_NUMBER"),col("CS_QUANTITY"),col("CS_WHOLESALE_COST"),col("CS_LIST_PRICE"))
#df_sort2.show()
```

Using **AGG Method** to aggregate the data

```
import snowflake.snowpark.functions as f

#.. Using AGG Method to aggregate the data

df_sum1 = df_tbl_read.agg(f.sum("CS_QUANTITY"))
df_stddev1 = df_tbl_read.agg(f.stddev("CS_QUANTITY"))
df_count1 = df_tbl_read.agg(f.count("CS_QUANTITY"))
df_max_min1 = df_tbl_read.agg(("CS_QUANTITY","min"),("CS_WHOLESALE_COST","max"))
df_max_min2 = df_tbl_read.agg({"CS_QUANTITY":"min",
                               "CS_WHOLESALE_COST":"max"})
#df_sum1.show() -- using f.<>
#df_stddev1.show() -- using f.<>
#df_count1.show() -- using f.<>
#df_max_min1.show() -- using Tuple()
#df_max_min2.show() -- using Dict{}
```

Using **Group_by** for grouping aggregate results

```
import snowflake.snowpark.functions as f

#.. Using Group_by for grouping aggregate results

df_group = df_tbl_read.group_by("CS_SOLD_DATE_SK").agg((f.sum("CS_QUANTITY").alias("QTY_SUM")),
f.stddev("CS_WHOLESALE_COST").alias("STD_Dev"))
#df_group.show()
```

Using **Window Method** as Window Function

```
#.. Using Window Method as Window Function

df_window = df_tbl_read.with_column("Rank", row_number().over(Window.order_by(col(
"CS_SOLD_DATE_SK").desc()))).select(col("Rank"),col("CS_ORDER_NUMBER"),col("CS_QUANTITY"),col(
"CS_WHOLESALE_COST"),col("CS_LIST_PRICE"))
#df_window.show()
```

Using **DataFrame NA Function** for Handling Missing Values

```
#.. Using DataFrame NA Function for Handling Missing Values

df_null_handling = df_tbl_read.filter((col("CS_QUANTITY").isNull()) & (col("CS_WHOLESALE_COST").
isNull())).sort([col("CS_QUANTITY"),col("CS_WHOLESALE_COST").desc()],ascending=[1,1]).select(col(
"CS_ORDER_NUMBER"),col("CS_QUANTITY"),col("CS_WHOLESALE_COST"),col("CS_LIST_PRICE"))
#df_null_handling.na.fill({"CS_QUANTITY":111}).show()
```

Performing an action on a DataFrame

Using the **Collect() Method** to generate an array of rows from a query

```
#df_tbl_read.collect()
```

Using **Collect_NoWait()** to Execute Query Asynchronously

```
df_async = df_tbl_read.collect_nowait()
#df_async.result()
```

Using Show Method to print the result

```
#df_tbl_read.show(5)
```

READING FROM S3 CSV

FOR CSV FILE WE HAVE TO PROVIDE SCHEMA using structtype and structfield

```
session = Session.builder.configs(connection_parameters).create()

schema = StructType([StructField("FIRST_NAME", StringType()),
StructField("LAST_NAME", StringType()),
StructField("EMAIL", StringType()),
StructField("ADDRESS", StringType()),
StructField("CITY", StringType()),
 StructField("DOJ",DateType())])

# Use session.read.schema and session.read.csv and mention the command to read data from s3
employee_s3 = session.read.schema(schema).csv('@my_s3_stage/employee/')
employee_s3.show()
employee_s3 = session.read.options({"ON_ERROR":"CONTINUE"}).schema(schema).csv(
'@my_s3_stage/employee/')
employee_s3.show()
type(employee_s3)

employee_s3 = employee_s3.cache_result()
employee_s3.is_cached

employee_s4=employee_s3.cache_result()
```

READING FROM S3 json

Note: employee_s3_json.cache_result() this command will throw error for semi structure data .. it will work for only csv data

We have to use select_expr

```
session = Session.builder.configs(connection_parameters).create()

# Mention command to read data from '@my_s3_stage/json_folder/'
employee_s3_json = session.read.json('@my_s3_stage/json_folder/')
employee_s3_json.show()
employee_s3_json.cache_result()###it wont work for semistructure data

employee_s3_json = employee_s3_json.select(col("$1").as_("new_col")).show()###it wont work

employee_s3_json = employee_s3_json.select_expr("$1:author","$1:id","$1:cat")
employee_s3_json.cache_result()

employee_s3_json.schema
```

SNOWPARK WRITE OPERATIONS

WE USE save_as_table COMMOND TO WRITE THE DATA INTO SNOWFALKE TABLE

Write has different modes. Overwrite will create oe replace table evry time

Mode APPEND will add data .. i.e it will insert data into existing table

```
# Write csv data from s3 to snowflake.

schema = StructType([StructField("FIRST_NAME", StringType()),
StructField("LAST_NAME", StringType()),
StructField("EMAIL", StringType()),
StructField("ADDRESS", StringType()),
StructField("CITY", StringType()),
 StructField("DOJ",DateType())])

employee_s3_csv = session.read.schema(schema).csv('@my_s3_stage/employee/')
employee_s3_csv.show()
employee_s3_csv = session.read.options({"ON_ERROR":"CONTINUE"}).schema(schema).csv(
'@my_s3_stage/employee/')
employee_s3_csv.write.mode("append").save_as_table("DEMO_DB.PUBLIC.EMPLOYEE_CSV")
```